

**DEVELOPMENT OF A WEB-BASED ANOMALY
DETECTION SYSTEM FOR E-LEARNING ENVIRONMENTS
USING MACHINE LEARNING ALGORITHMS**

BY

**ARCHIBONG, OROK INAMETI
(22CG031837)**

**A PROJECT SUBMITTED TO THE
DEPARTMENT OF COMPUTER AND INFORMATION
SCIENCES, COLLEGE OF SCIENCE AND TECHNOLOGY,
COVENANT UNIVERSITY OTA, OGUN STATE.**

**IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR
THE AWARD OF THE BACHELOR OF SCIENCE
(HONOURS) DEGREE IN COMPUTER SCIENCE.**

JUNE, 2026

CERTIFICATION

I hereby certify that this project was carried out by ARCHIBONG, Orok Inameti in the Department of Computer and Information Sciences, College of Science and Technology, Covenant University, Ogun State, Nigeria, under my supervision.

Dr. Oluranti Jonathan

Supervisor

Signature and Date

Prof. Aderonke Oni

Head of Department

Signature and Date

DEDICATION

This project is dedicated to Almighty God for His guidance, wisdom, and strength throughout this journey. I also dedicate this work to my beloved parents, Mr and Mrs Emmanuel Archibong, whose unwavering support, encouragement, and sacrifices have been my greatest motivation.

ACKNOWLEDGEMENTS

I wish to express my profound gratitude to my supervisor, Dr. Jonathan Oluranti, for his invaluable guidance, patience, and constructive feedback throughout the development of this project. Your expertise and dedication have been instrumental in shaping this work.

I also acknowledge the support of the Department of Computer and Information Sciences, Covenant University, for providing the resources and environment necessary for this research. My sincere appreciation goes to my colleagues and friends who provided moral support and technical insights during the development of this system. Finally, I am grateful to all the lecturers and academic staff who have imparted knowledge and skills that directly contributed to the successful completion of this project.

TABLE OF CONTENTS

CONTENT	PAGES
COVER PAGE	i
CERTIFICATION	ii
DEDICATION	iii
ACKNOWLEDGEMENTS	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	ix
LIST OF TABLES	x
ABBREVIATIONS	xi
ABSTRACT	xiii
CHAPTER ONE	1
INTRODUCTION	1
1.1 Background to the Study	1
1.2 Statement of the Problem	1
1.3 Aim and Objectives of the Study	2
1.4 Methodology	3
1.5 Significance of Study	6
1.6 Scope	6
1.7 Project Outline	7
CHAPTER TWO	8
LITERATURE REVIEW	8
2.1 Preamble	8
2.2 Overview of E-Learning and Academic Integrity	8
2.2.1 Definition and Growth of E-Learning	8
2.2.2 Academic Integrity in Online Examinations	8
2.2.3 Challenges of Maintaining Integrity in Online Examinations	10
2.3 Anomaly Detection in Educational Contexts	10
2.3.1 Definition and Taxonomy of Anomaly Detection	12
2.3.2 Types of Anomalies	12
2.4 Data Processing Pipelines in Educational Technology	13
2.5 Machine Learning for Anomaly Detection	13
2.5.1 Categories of Machine Learning	14

2.5.2 Unsupervised Learning	15
2.5.3 Isolation Forest Algorithm	15
2.5.4 Autoencoder Neural Networks	17
2.6 Big Data in E-Learning Systems	19
2.6.1 Big Data Theory (3Vs, 5Vs, 7Vs Models)	20
2.6.2 Outlier Theory	22
2.6.3 Behavioural Analytics Theory	22
2.6.4 Feature Engineering for Examination Behaviour	22
2.6.5 Comparative Analysis of Isolation Forest and Autoencoder	24
2.6.6 Empirical Studies on Big Data in Educational Systems	24
2.6.7 Empirical Studies on Anomaly Detection in E-Learning	25
2.6.8 Empirical Use of Machine Learning Models for Anomaly Identification	26
2.6.9 Empirical Studies on Data Pipelines for Real-Time Analytics	26
2.7 Review of Existing Systems	27
2.7.1 Commercial Online Proctoring Solutions	27
2.7.2. Learning Management System Integrity Features	27
2.8 Summary	27
CHAPTER THREE	29
SYSTEM ANALYSIS AND DESIGN	29
3.1. Preamble	29
3.2. Requirement Analysis	29
3.2.1. Functional Requirements	29
3.2.2. Non-Functional Requirements	30
3.3. System Architecture	31
3.4 Physical Design	34
3.4.1 Machine Learning Pipeline	34
3.4.2 Behaviour Tracking Module	34
3.5 Logical Design	34
3.5.1 Sequence Diagram	35
3.5.2 Class Diagram	36
3.5.3 Use-Case Diagram	37
3.5.4 Activity Diagram	38
3.5.5 Entity Relationship Diagram	40

3.6 Evaluation of the System	40
3.6.1 Algorithm Evaluation Criteria	41
3.6.2 Statistical Analysis Method	41
CHAPTER FOUR	42
SYSTEM IMPLEMENTATION AND EVALUATION	42
4.1 Preamble	42
4.2 System Requirements	42
4.2.1 Hardware Requirements	42
4.2.2 Software Requirements	43
4.3 Implementation Tools	43
4.3.1 Python	43
4.3.2 Flask	43
4.3.3 SQLAlchemy	43
4.3.4 scikit-learn	44
4.3.5 TensorFlow and Keras	44
4.3.6 NumPy and pandas	44
4.3.7 joblib	44
4.3.8 Chart.js	44
4.3.9 Visual Studio Code	44
4.4 Development Methodology	44
4.5 Evaluation of the System	44
4.5.1 Algorithm Performance Evaluation Results	45
4.5.2 User Satisfaction Evaluation Results	48
4.5.3 Discussion	48
4.6 Program Modules and Interfaces	49
4.6.1 The Login Module	49
4.6.2 The Examiner Dashboard Module	50
4.6.3 The Examination Interface	51
4.6.4 The Examination Results Interface	52
4.6.5 The Algorithm Evaluation Dashboard	53
4.6.6 The Examination Management Module	55
CHAPTER FIVE	57
Summary, Recommendations, and Conclusion	57

5.1 Summary	57
5.2 Recommendations for Future Work	57
5.3 Conclusion	59
REFERENCES	60

LIST OF FIGURES

FIGURE	TITLE OF FIGURE	PAGES
2.1	Architecture of a Typical E-Learning Platform	10
2.2	Work flow of the isolation Forest Algorithm	12
2.3	Autoencoder Architecture for Anomaly Detection	14
2.4	Importance of Big Data in E-learning	24
3.1	System Architecture	33
3.2	Sequence Diagram of the ExamGuard System	30
3.3	Class Diagram of the ExamGuard System	37
3.4	Use-Case Diagram of the ExamGuard System	38
3.5	Activity Diagram of the ExamGuard System	39
3.6	Entity Relationship Diagram of ExamGuard	45
4.1	Isolation Forest Score Distribution Chart	48
4.2	Autoencoder Reconstruction Error Distribution Chart	48
4.3	ROC Curve Comparison — Isolation Forest vs Autoencoder	49
4.4	Student Login Interface	51
4.5	Examiner Dashboard Interface	52
4.6	Exam Interface with Behaviour Monitoring	53
4.7	Exam Results Interface	54
4.8	Algorithm Evaluation Dashboard	55
4.9	Algorithm Evaluation Dashboard 2	55
4.10	Manage Module Dashboard	56
4.11	Add Questions Dashboard	57
4.12	Create Exam Dashboard	57

LIST OF TABLES

TABLE	TITLE OF TABLE	PAGES
1.1	Objectives-Methodology Mapping Table	4
2.1	Types of Anomalies	12
2.2	Types of Machine Learning with examples	14
2.3	Behavioural Features Used for Anomaly Detection	24
3.1	Functional Requirements of ExamGuard	29
3.2	Non-Functional Requirements of ExamGuard	30
4.1	Hardware Requirements Table	42
4.2	Software Requirements Table	43
4.3	Algorithm Performance Metrics Comparison	45
4.4	Evaluation Task	48

ABBREVIATIONS

AE	Autoencoder
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
DB	Database
ERD	Entity-Relationship Diagram
FN	False Negative
FP	False Positive
HTML	Hyper-Text Markup Language
IF	Isolation Forest
JSON	JavaScript Object Notation
MAS	Multi-Agent Systems
ML	Machine Learning
MSE	Mean Squared Error
NLP	Natural Language Processing
OULAD	Open University Learning Analytics Dataset
PSSUQ	Post-Study System Usability Questionnaire
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SQL	Structured Query Language
TN	True Negative
TP	True Positive
UI	User Interface

UML Unified Modelling Language

ABSTRACT

The proliferation of online learning platforms has introduced significant challenges to academic integrity, particularly in monitoring student behaviour during online examinations. Traditional invigilation methods are inadequate for detecting anomalous behaviour in remote examination environments, creating opportunities for academic misconduct. This study presents ExamGuard, an anomaly detection system for e-learning environments that employs two unsupervised machine learning algorithms — Isolation Forest and Autoencoder — to monitor and flag suspicious student behaviour during online exams. The system tracks six behavioural metrics: examination score, total mouse clicks, time spent on the exam, tab switches, average time per question, and activity rate. Both algorithms were trained on synthetic behavioural datasets comprising 1,000 normal and 100 anomalous samples to establish baselines for normal student behaviour. A Flask-based web application provides the backend infrastructure, while a responsive frontend facilitates examination delivery, real-time behaviour monitoring, and an examiner dashboard. A comparative evaluation of both algorithms was conducted using quantitative metrics including Precision, Recall, F1 Score, Area Under the ROC Curve (AUC), and Silhouette Score, providing examiners with evidence-based insights to determine the more suitable algorithm for deployment. Evaluation using a modified Post-Study System Usability Questionnaire (PSSUQ) indicated satisfactory user experience. The study demonstrates the feasibility and effectiveness of unsupervised machine learning in detecting behavioural anomalies in e-learning environments, offering a scalable and automated solution to academic integrity challenges.

Keywords: Anomaly Detection, E-Learning, Isolation Forest, Autoencoder, Academic Integrity, Behavioural Monitoring, Machine Learning, Online Examination

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

The use of e-learning platforms, like Moodle, Coursera and Google Classroom, has increased enormously in educational institutions in the past decade thanks to the knowledge dissemination flexibility and scalability they offer. These systems have facilitated the delivery of learning material to millions of learners at a distance. These platforms collect vast amounts of data as students log in, engage with content, take assessments, and utilize the platform. These platforms create significant amounts of data as students log in, interact with content, take assessments, and otherwise use the platform. This has led to the birth of the field called Learning Analytics (LA) which involves the use of data analysis methods to monitor and enhance learning processes (Akçapınar, Altun, & Aşkar, 2019).

One of the challenges in modern e-learning environments is to detect irregular or anomalous behaviors, however. These anomalies range from poor performance, several logins from various locations, academic dishonesty, low engagement, and other irregularities in participation. These kinds of activities, if they exist, can be detected manually only with effort and virtually never when data grows exponentially. Hence the importance of an automated and intelligent system that can detect these abnormalities in real-time (Doulat, & Maher, 2023). Conventional anomaly detection methods are often rule-based and static, which are not effective to detect anomalies in diverse and voluminous data produced in educational systems. Furthermore, a lot of the current approaches have been developed to work with limited amounts of data and do not have enough computational resources to handle the real-time data streams emitted by thousands of learners. The limitation has led to a gap in the ability to integrate the Big Data technologies and Machine Learning algorithms which can efficiently process and accurately analyze massive data sets.

To overcome these challenges, the authors proposed a Big Data Anomaly Detection Pipeline, which integrates several components: data ingestion, data preprocessing, machine learning, and visualization into a single pipeline (Huang, 2025). A scalable and efficient solution can be created using technologies like Apache Kafka (data streaming), Apache Spark (distributed processing), and machine learning algorithms such as Isolation Forest and Autoencoder. This kind of pipeline can automatically identify abnormal student use, alert, and real-time visualization of insights via dashboards.

The use of this system in e-learning platforms would help educate and administrators to better understand student behavior, detect early signs of at-risk learners, prevent academically dishonest students, and improve learning experiences. This project also illustrates the real-world application of the principles and methodologies of the three pillars of Big Data Engineering, Artificial Intelligence, and Educational Data Mining, bringing the concepts of data-driven technologies and contemporary education together.

1.2 Statement of the Problem

In the era of the development of online education platforms, a large volume of data is created daily, such as students' log-in time, quiz attempts, learning time, click behavior, and assessment results (Yamasari, Qoiriah, Rochmawati, Prapanca, Prihanto, Suartana, Putra, 2024). However, conventional monitoring systems in LMS do not support mass data or real-time data streams, which means it is challenging to detect anomalies in time like academic dishonesty, automated bot participation, sudden performance gains or drops, inactive or disengaged learners, etc. (Al Doulat & Maher, 2023). The lack of automated anomaly detection impacts the integrity of assessments, and overall, the educational quality, therefore, the introduction of an anomaly detection system in this context is crucial.

The current online proctoring programs are largely based on video surveillance, keystroke logging, and eye tracking. Such methods involve considerable privacy issues for students and teachers and may involve installing special software, using webcams and microphones, and collecting biometric information. Moreover, the cost of such systems can be high for many educational institutions especially in low-resource areas and the technical requirements are not feasible for the students in resource-poor settings.

More importantly, current systems are unable to utilise unsupervised machine learning to explore the inherent behavioural patterns of students while taking tests. The real-world lack of automatic learning baseline normal behaviour and cheat detection systems that do not require labelled training data and human pre-configuration of cheat detection rules is significant. In addition, there is no comparison or contrast between how well various anomaly detection algorithms perform in practical examination systems, which prevents administrators from having objective evidence to choose an algorithm.

To overcome these deficiencies, this research aimed at creating ExamGuard, a web-based e-learning anomaly detection system based on Isolation Forest and Autoencoder algorithms for the purpose of studying the behaviours of students during online examination. In addition to being transparent and

private, the system is also inexpensive, and gives an integrated evaluation framework for comparing the performance of the two algorithms.

1.3 Aim and Objectives of the Study

This study aims to design and develop a Web-Based Anomaly Detection Pipeline capable of automatically identifying irregular or suspicious patterns in student activities on e-learning platforms using machine learning and real-time data processing technologies. The objectives include;

- i. To review existing systems and literature on anomaly detection in E-learning platforms and identify their limitations.
- ii. To collect, clean, and process e-learning data for model training and testing.
- iii. To train the model using machine learning techniques.
- iv. To design and implement an end-to-end big anomaly detection system.
- v. To evaluate the system's performance using specific metrics.

1.4 Methodology

The following are the methodologies used in this project:

- i. To review existing systems and literature on anomaly detection in E-learning platforms and identify their limitations.**

Conduct a comprehensive literature review on existing anomaly detection systems, datasets, and technologies used in learning analytics. Also evaluate other anomaly detection systems to detect gaps and problems with them.

- ii. To collect, clean, and process e-learning data for model training and testing.**

Collect, clean, and process e-learning data for model training and testing. This is where all the necessary data will be collected and, in this case, the data will be retrieved from the Open University Learning Analytics Dataset (OULAD). After the data has been collected, it will be cleaned and processed manually by removing duplicated events and missing timestamps.

- iii. To train the model using machine learning techniques.**

Train two unsupervised machine learning models: Isolation Forest and Autoencoder. The Isolation Forest model will be trained to identify anomalies by isolating unusual data points through random partitioning, while the Autoencoder will be trained to learn normal behavioral patterns and detect anomalies using reconstruction error. Model parameters will be configured

and optimized to improve detection performance before integrating the trained models into the ExamGuard system.

iv. To design and implement an end-to-end big data anomaly detection system.

Design and build an anomaly detection system. The front end will be built using HTML, CSS and JavaScript for user interaction while the backend will be built with the Flask framework. SQLite will be used as the database management system. REST APIs will be integrated to facilitate communication between the frontend and the backend components.

v. To evaluate the system's performance using specific metrics.

Evaluate model and system performance using accuracy metrics like Precision, Recall, F1-Score, ROC AUC, and Silhouette score.

Table 1.1 shows a mapping of the various objectives of this project and the methodologies used to achieve them in a tabular format.

Table 1.1: Objectives-Methodology Mapping Table

S/N	OBJECTIVES	METHODOLOGIES
1	To examine existing systems and literature on anomaly detection in e-learning platforms and identify their limitations, particularly in real-time detection and scalability.	i. Perform a detailed literature review on existing anomaly detection systems, datasets, and technology used in learning analytics. Examine the tools, methods and limitations. ii. Evaluating existing anomaly detection systems to look for gaps/ problems.
2	To collect, clean, and process e-learning data for model training and testing,	i. Download and organize the Open University learning Analytics Dataset. ii. Load the dataset into my Python environment using Pandas

		iii. Clean and pre-process the text (remove wrong, inconsistent, duplicated, or missing data). iv. Convert logs into useful features. Score, total clicks, time on exam, tab switches, average time per question, and activity rate are the key metrics required for this project
3	To train the model using machine learning techniques	i. Train two unsupervised machine learning models: Isolation Forest and Autoencoder. ii. The Isolation Forest model will be trained to identify anomalies by isolating unusual data points through random partitioning, while the Autoencoder will be trained to learn normal behavioral patterns and detect anomalies using reconstruction error. iii. Model parameters will be configured and optimized to improve detection performance before integrating the trained models into the ExamGuard system.
4	To design and implement an end-to-end web-based anomaly detection system.	i. The data set is divided into training and testing sets. The

		data model is then trained using Isolation Forest and Autoencoder algorithms to detect outliers. ii. Build the front end using HTML, CSS and JavaScript to display anomaly results. iii. Develop the backend with Flask framework. iv. Use SQLite for data storage. v. Implement the system by integrating the trained model into the backend framework.
5	To evaluate the system's performance using specific metrics.	i. Test the model's predictions against manual supervision for efficiency. ii. Evaluate usability with student and staff feedback. iii. Measure system efficiency, such as response time and scalability.

1.5 Significance of Study

This study has several noteworthy contributions to the e-learning, academic integrity and applied machine learning arenas:

- i. It shows how to apply unsupervised machine learning algorithms in real-time online exam monitoring, which improves the solution by avoiding the financial investment and privacy concerns of proctoring solutions.
- ii. It brings a comparative evaluation model for the Isolation Forest and Autoencoder algorithms in the context of an operational e-learning environment, supporting empirical evidence for the selection of algorithms.

- iii. It tackles one of the difficulties in most real-world academic integrity applications, namely the lack of labeled training data.
- iv. It is open source and accessible, and offers an easily scaled template for institutions interested in incorporating behavioural analytics into their e-learning systems.
- v. It promotes the understanding of behavioural cues that can be used to identify problematic exam behavior, helping to shape more sophisticated academic integrity policies.

1.6 Scope

This project is restricted to the collection, processing, analysis and visualization of the data about the interaction of the learner with the use of Big Data technologies and machine learning techniques. The system will generate or be able to import logs from an e-learning platform (Moodle, Canvas, etc.) and look for any irregularities, like sudden inactivity, multiple logins from different locations, irregular submission times, or abnormal variations in scores. The data that will be used will be mainly the engagement metrics of students (such as login time, duration, quiz attempts, submission frequency, page visits, etc.).

1.7 Project Outline

The study is presented by giving background and motivation, stating the problem, aim and objectives, research methodology, significance of the study and structure of the study in Chapter One. Chapter Two reviews broadly the literature on e-learning, academic integrity, anomaly detection, and the particular machine learning algorithms used in this study. It also evaluates the current systems and pinpoints the gaps which this project will fill. Chapter Three outlines the System Analysis and Design consisting of functional and non-functional requirements, system architecture, physical and logic design and evaluation framework used. The implementation of the ExamGuard system is explained in Chapter Four, starting with the system requirements, followed by the implementation tools, the program development methodology, the evaluation results and descriptions of the program modules and interfaces. The project is summarized in Chapter Five, and recommendations for future development and a conclusion are presented.

CHAPTER TWO

LITERATURE REVIEW

2.1 Preamble

The chapter summarizes the current research work done in the area of anomaly detection in e-learning environments with special attention to the theoretical, methodological and practical aspects pertinent to the ExamGuard system. The chapter addresses the history of e-learning and relevant Academic Honesty issues, takes a closer look at the concepts and application of anomaly detection in educational settings, and examines the machine learning algorithms currently used and their limitations to inspire this work.

2.2 Overview of E-Learning and Academic Integrity

Ever took a test which you studied very hard for, but not been able to get the outcome you expected? This test result has a PROBLEM, an ANOMALY. E-learning has become a backbone in global education and it is very important to implement some measures to ensure the validation of the education and those measures are the birth of the concept of E-learning anomaly detection system. Some of the biggest issues with online education are cheating in online quizzes, Sudden drops in learner engagement, Unpredictable behavior patterns, certification fraud, and misleading performance analysis (Tricia et al,2026). These issues call for automated systems that can analyze student data as it happens, and identify anomalies that may indicate deeper problems. The size, rate and complexity of e-learning logs of behaviour are not well supported by traditional anomaly detection techniques. Thus, this project is a validation of the development of a Big Data Anomaly Detection Pipeline, tailored to process large and dynamic set of students activity.

2.2.1 Definition and Growth of E-Learning

Since its beginnings in the 1990s, e-learning has experienced significant change in its definition and in the ways that educational content and activities are delivered via electronic media and Internet (Moore et al., 2011). While today's e-learning environments began with static web pages and downloadable course materials, they now include interactive learning management systems, real-time video lectures, adaptive learning platforms and advanced assessment tools. The market for e-learning is estimated to be roughly USD 250 billion in 2020 and is expected to reach USD 1 trillion by 2027, demonstrating the vital role played by e-learning in modern education (Global Market Insights, 2021). The COVID-19 pandemic has forced educational institutions, worldwide, to rapidly shift to e-learning

within weeks. This quick shift highlighted the capability of the current e-learning framework and its constraints, especially when it comes to the integrity of assessments. It was much easier to provide lectures and course materials online, but creating the integrity protections normally provided in a classroom setting was more difficult.

2.2.2 Academic Integrity in Online Examinations

Academic integrity is defined as the obligation to engage in honest, trustworthy, fair, respectful and responsible behavior in all academic activity (International Center for Academic Integrity, 2021). Academic honesty in an examination setting relates to the belief that every student's performance is determined by their own knowledge and ability, not by improper use of others' knowledge and ability. The integrity issues in online examinations are different from those in traditional examinations or are at least much more pronounced in online examinations.

Key types of academic dishonesty observed in online exams include: cheating using unauthorized digital resources (websites, documents or an online calculator); communicating with peers through messaging during an exam; using academic contract cheating services to complete assignments for students; sharing information about exams with someone else who will take the test later (Tricia et al., 2026). Studies have shown that students are more likely to cheat on on-line exams than on exams taken in a physical classroom. Harmon and Lambrinos (2008) discovered that, in unproctored online exams, students performed significantly better than their in-person, proctored counterparts, indicating systematic cheating in unproctored exams. This is a reason that much of this cheating is believed to be occurring in the online setting.

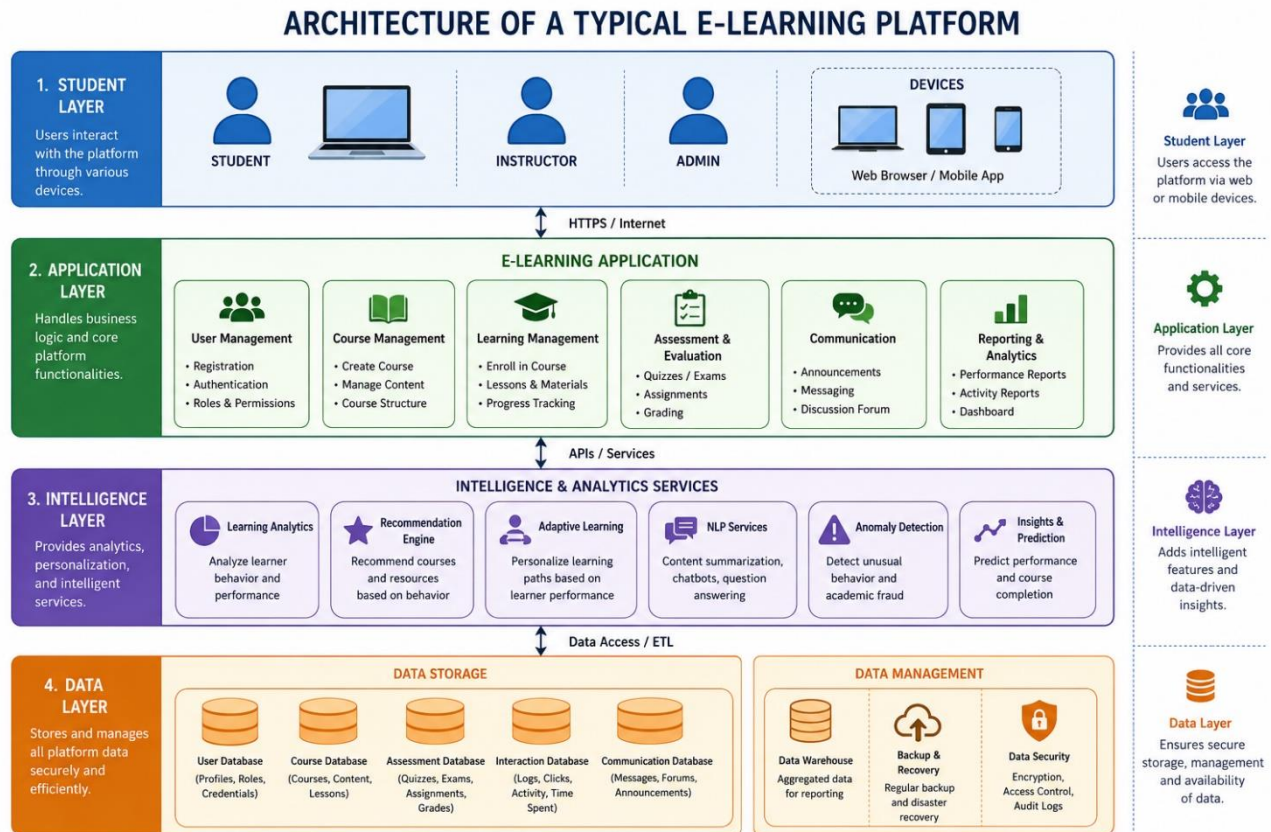


Figure 2.1: Architecture of a Typical E-Learning Platform

2.2.3 Challenges of Maintaining Integrity in Online Examinations

Academic integrity is difficult to maintain in an online examination setting because of a number of factors. There are no physical invigilators, which eliminates one of the major deterrents of bad behavior in conventional exams. Students are able to examine from an environment where access to resources, devices and/or other people cannot be controlled. Examinations take place in a variety of settings, from home to public space, creating variations and making it challenging to monitor standardised examinations.

Use of lockdown browsers is a technical solution that tries to limit students' access to other applications during exams however, students have found ways to circumvent these limitations, including using a second device. More comprehensive, video-based proctoring systems raise privacy concerns and pose technical challenges for students with limited internet bandwidth or without access to webcams (Luo, 2024). The challenges highlight the importance of non-surveillance or behavioural monitoring methods that are light on the system and privacy-friendly.

2.3 Anomaly Detection in Educational Contexts

Irregular or suspicious behavior may be identified more easily in a traditional classroom environment by observing these behaviors directly. In e-learning settings, however, interactions are indirect, in the form of system logs and data traces. Therefore, anomalous behaviour is not something which can be directly observed, but rather something which is identified as being abnormal as it is apart from statistical, behavioural or structural norms derived from large-scale digital logs (Romero & Ventura, 2020). Anomalous behavior is any interaction that falls outside the norm or what is expected for a student (Romero & Ventura, 2022). In e-learning systems, abnormalities can be manifested in different ways, such as the quick completion of a quiz with a high percentage correct, which is a sign of cheating, multiple logins around the same time from different geographical locations, bot-like automated logins, or extended periods of unusual inactivity before assessments (Zhang, Li & Zhao, 2021). Identifying these anomalies is crucial for upholding academic honesty and for ensuring accurate learning analytics. The lack of detecting anomalies in e-learning datasets can affect the fairness of assessment, impair the trustworthiness of an institution and prevent appropriate pedagogical intervention (see also Alzahrani 2021). Several reasons why there is a need to detect anomalies:

i. Ensuring Academic Integrity

Cheating, impersonation, and the misuse of resources is becoming an increasing problem with online assessments. Anomalous behaviour detection aids institutions to ensure they are fair and credible (Henriksen, Creely, & Henderson, 2020).

ii. Improving Instructional Decision-Making

Anomalies can signal non-engaged students or students who are at risk of not being successful in school. Early detection means intervention at an opportune time.

iii. Enhancing System Security

Content may be attempted to be extracted or learning activities disrupted by bots, and by malware scripts and unauthorized users.

iv. Optimizing System Performance

Anomaly detection can aid system bug detection, misconfiguration, or performance bottlenecks identification.

2.3.1 Definition and Taxonomy of Anomaly Detection

Anomaly detection can aid system bug detection, misconfiguration, or performance bottlenecks identification. According to Bittle & El-Gayar the term anomaly in general sense refers to data that is unusual and does not fit with the majority of data (or the expectations made on the basis of the majority of data). Anomalies can be identified with statistical and machine learning techniques, and in various types of data, such as images and time series data (Malhotra et al., 2015). An anomaly is an observation, event or piece of data that is considerably different from what is expected or observed in a system. It is a pattern that is not representative of most data and usually means something unusual, something wrong, something that is noteworthy has happened. Depending on the context, an anomaly is also known as an outlier, exception, or abnormal pattern. Anomaly detection is one of the simplest use cases of unsupervised learning, the goal of which is to predict anomalies, bottlenecks or security breaches beforehand (Usama et al., 2017). Some of the important properties of anomalies are:

- i. It is uncommon or rare. It should happen very rarely in comparison to the normal data pattern.
- ii. It is unexpected. It is not consistent with the standard behaviour or trends of the system.
- iii. It is influential. Anomalies can contain valuable data, as they can represent faults or errors in the system, fraudulent activities, breaches in security, abrupt or hazardous changes.
- iv. It is context-dependent. An anomaly is the opposite of what is normal in one situation, and normal in another.

2.3.2 Types of Anomalies

Anomalies can be of different types, like:

- i. Point Anomaly: This is a point that does not fit the other data points.
- ii. Contextual Anomaly: This is an anomaly that is only abnormal in a given context. The abnormality of the anomaly is dependent on the context.
- iii. Collective Anomaly: When a set of related data points are in an unusual pattern.

Table 2.1: Types of Anomalies

Types of Anomalies	Definition	Examples
Point Anomaly	This is where a single point stands out from the rest of the data set. An example is when an individual makes a transaction of 500 million Naira when the previous transactions have been in tens of thousands.	When an individual makes a transaction of 500 million Naira when the previous transactions have been in tens of thousands.
Contextual Anomaly	This is when a data point is only abnormal in a specific context. The abnormality of the anomaly is dependent on the context.	A temperature of 30°C is normal during the dry season but very abnormal during the rainy season.
Collective Anomaly	This is when a group of related data points form an unusual pattern.	A sudden group of failed login attempts can be inferred to be a cyberattack.

Detecting anomalies early is very important because it helps detect security and firewall attacks, and it helps save costs, especially when detected on time.

2.4 Data Processing Pipelines in Educational Technology

A pipeline is a multi-stage computational structure that processes data sequentially, from raw data acquisition to meaningful output. In the field of educational technology, pipelines are usually composed of data collection, data preprocessing, feature engineering, model training, and visualization of the results (Al-Doulat & Maher, 2023). Nisar and Tariq (2022) state that data pipelines help achieve scalability, consistency and efficiency when working with large volumes of data. In the context of e-learning systems, pipelines provide continuous monitoring and detection of irregular activities, an aspect that is crucial in such a dynamic and event-driven environment since the interactions that students have with the system are dynamic in nature and follow an event-driven paradigm (Nisar & Tariq, 2022). The pipeline model also works for both batch and streaming processes, suitable for large-scale learning management systems.

2.5 Machine Learning for Anomaly Detection

Machine learning is a subset of artificial intelligence (AI) which involves the ability of machines to learn patterns from data and make decisions or predictions without any explicit programming. It questions and explains why learning algorithms work, what their performance potential is on new data, and what are the limitations of what a learner can learn. ML is one of the fundamental disciplines that allows computer systems to learn from vast amounts of data without being explicitly programmed (Kiura et al., 2023). They are all related to the building of the models and engineering the systems, while ML theory formalizes the problem, provides performance bound, and explains trade-offs (data vs. error, model complexity vs. generalization, computation vs. accuracy). The basis of Machine Learning is to gather data, and then input the gathered data into an algorithm. The algorithm then identifies patterns, the model makes predictions, and the model gets better with time.

Machine Learning (ML) is one of the most important subfields of Artificial Intelligence (AI) and plays a vital role in today's data processing, especially in complex areas like networking and e-learning (Kiura et al., 2023; Usama et al., 2017). Anomaly detection in complex datasets using machine learning (ML) has long been a known method. In the ML-based anomaly detection models, the algorithms learn what is considered "normal" behavior and detect deviations that are "anomalous" in terms of the learned patterns. In practice, the algorithms: Isolation Forest (Liu, Ting and Zhou, 2012), One-Class SVM (Chandola, Banerjee and Kumar, 2009), Autoencoders and Clustering-based techniques, have been used extensively to solve anomaly detection problems in various applications. In an e-learning environment, these algorithms are used to detect problematic behavior, such as cheating, academic dishonesty, system abuse, or lack of learning engagement (Qiu et al., 2021). The flexibility of the ML models allows them to be applied to the very heterogeneous and dynamic nature of e-learning data.

2.5.1 Categories of Machine Learning

Over time, these algorithms gain 'intelligence' as more data is fed into them, which allows the algorithms to learn and optimize their work. Machine learning algorithms are of four types: Supervised Learning, Semi-Supervised Learning, Unsupervised Learning, and Reinforcement Learning. The present study is more in the line of unsupervised learning theory since the aim is to identify unusual behavior without the presence of a class of labeled instances of fraudulent or irregular behavior. Hastie, Tibshirani, and Friedman (2009) state that unsupervised learning is used to discover hidden structures

within the data and it is best suited for anomaly detection applications in which anomalies are rare or the definition of an anomaly is not known.

Table 2.2: Types of Machine Learning with examples

Category	Description	Examples / Use Cases
Supervised Learning	The model is trained on labeled data, meaning each input has a corresponding output. The goal is to predict outputs for new inputs.	Classification: Email spam detection, student performance prediction. Regression: Predicting quiz scores, course completion time
Unsupervised Learning	The model is trained on unlabeled data. It finds patterns, structures, or anomalies without predefined outputs.	Clustering: Grouping students based on learning behavior Anomaly Detection: Detecting unusual activity in E-learning platforms
Semi-supervised Learning	Uses a mix of labeled and unlabeled data. Useful when labeling data is expensive or time-consuming.	Predicting student performance with limited labeled records Text classification with few labeled documents
Reinforcement Learning	The model learns by interacting with an environment and receiving feedback (rewards or penalties) for actions.	Adaptive tutoring systems that adjust content based on student responses Personalized learning paths

2.5.2 Unsupervised Learning

Unsupervised Learning: This is when machine learning models that analyze and categorize data on their own without any known label or category. The main goal is to discover elusive patterns, outlying points or natural clusters in the data (Jain, 2020). Clustering, dimensionality reduction and anomaly detection are common applications of unsupervised learning. One of the most important differences between UL and supervised learning is the lack of response variables (Gambella et al., 2019). The training set is only an input data set with no target output to be assigned (Kiura et al., 2023). It is very unlikely that it has a complete and well-defined data set of student activities that can be clearly

classified as normal and anomalous, such as in an e learning setting. Unsupervised learning does not need labelled data and will model normal behavior and mark deviations as anomalies. This is very close to the real world, where the type of the anomalies is not known or changes. The application of deep learning has proven to be a successful approach to anomaly detection recently, especially in high-dimensional and complex datasets. Autoencoders and Recurrent neural networks (RNNs) can be used to learn complex patterns of normal behavior and recognize deviations as anomalies (Chalapathy & Chawla, 2019). Deep learning has been used to identify abnormal clickstream sequences for detecting unusual user behaviour in the context of E-learning. It is important to note that for relatively small feature spaces and structured E-learning datasets, traditional unsupervised learning techniques, like Isolation Forest or clustering still work well and are not too expensive.

2.5.3 Isolation Forest Algorithm

Unlike most anomaly detection techniques, the Isolation Forest algorithm proposed by Liu et al. (2008) is a completely novel one. Instead of building a profile of normal data and detecting anomalies, Isolation Forest actually isolates anomalies by taking advantage of their distinguishing characteristics: Anomalies are rare and they are quite different from normal data points; Anomaly detection is easier based on random partitioning. The algorithm creates an ensemble of isolation trees constructed by choosing a randomly selected feature and randomly selected split value from the range of the selected feature (Alzahrani,2022). This process is repeated recursively until each instance comes to reside in a leaf node. The isolation depth, which is the number of splits needed to isolate an instance, is used as this anomaly score, with instances that are isolated with few splits having high anomaly scores, and

those that require many splits before isolation having normal anomaly scores (Choudhury & Sabharwal, 2023).

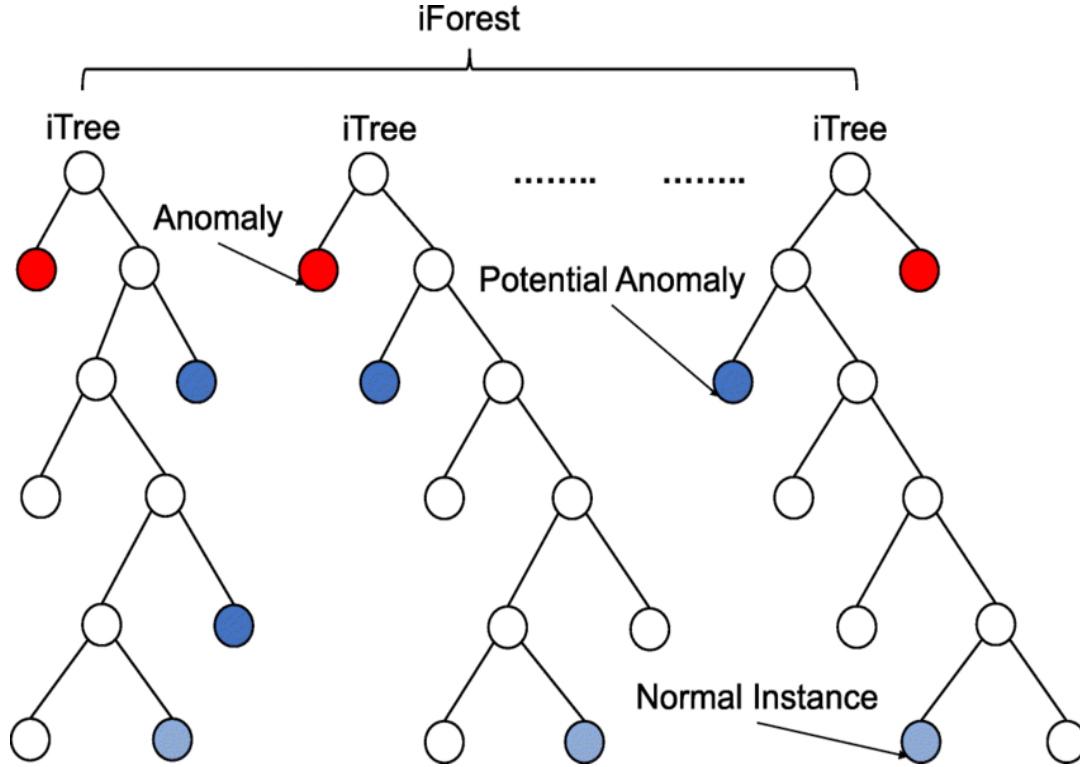


Figure 2.2: Work flow of the isolation Forest Algorithm

The key theoretical support for the effectiveness of Isolation Forest is the geometric characteristics of anomalous data. As anomalies are far from the majority of normal data, it is a safe bet that they are split apart by the first few random partitions that are visited. The normal instances are embedded in dense parts and will have to be isolated through many partitions. This property also makes Isolation Forest very efficient and can be easily scaled to large, high-dimensional data sets, because there is no need to compute distances or densities. The decision threshold is determined by the contamination parameter that defines the percentage of anomalies expected in the data set. The contamination value in ExamGuard system was set to 0.1 (10%) which corresponds to the ratio of potentially problematic exam sessions in the training set. The parameter `n_estimators` was set to 100, which is a good compromise between accuracy and computational time. The algorithm works with the six common behavioral features, and uses a `StandardScaler` to ensure that each feature is given equal weight in the isolation process. Isolation Forest has been shown to be an effective algorithm for various tasks of

anomaly detection. It has been shown to be superior to Local Outlier Factor and one-class SVM by Liu et al. (2012) on benchmark datasets. It is used in real-time deployments because its training time is $O(n \log n)$ and its scoring time is $O(t \log n)$, where t is the number of trees and n is the number of instances. Also, it is an advantage when interpretability is desired compared to neural network approaches..

2.5.4 Autoencoder Neural Networks

An autoencoder is a type of artificial neural network that, using unsupervised learning, extracts efficient representations from the data of its inputs (Al-Shabandar et al., 2024). It consists of two networks: an encoder network which encodes the input data to a lower dimensional latent representation, and a decoder network which reconstructs the original data from the latent representation. The network learns to produce a minimum reconstruction error, usually the mean squared error of the difference between the input and the reconstruction of the network

Autoencoders for anomaly detection take advantage of the network being specialized in the reconstruction of the normal patterns when the network is trained only with normal data. The autoencoder does not reconstruct the data accurately when the input data are abnormal, i.e., a specific type of data that is significantly different from the training data distribution, as the reconstruction error is high (Sae-Ang et al., 2024). The reconstruction error is then used as an anomaly score and those with errors above a given value are considered anomalous.

Autoencoder Architecture for Anomaly Detection

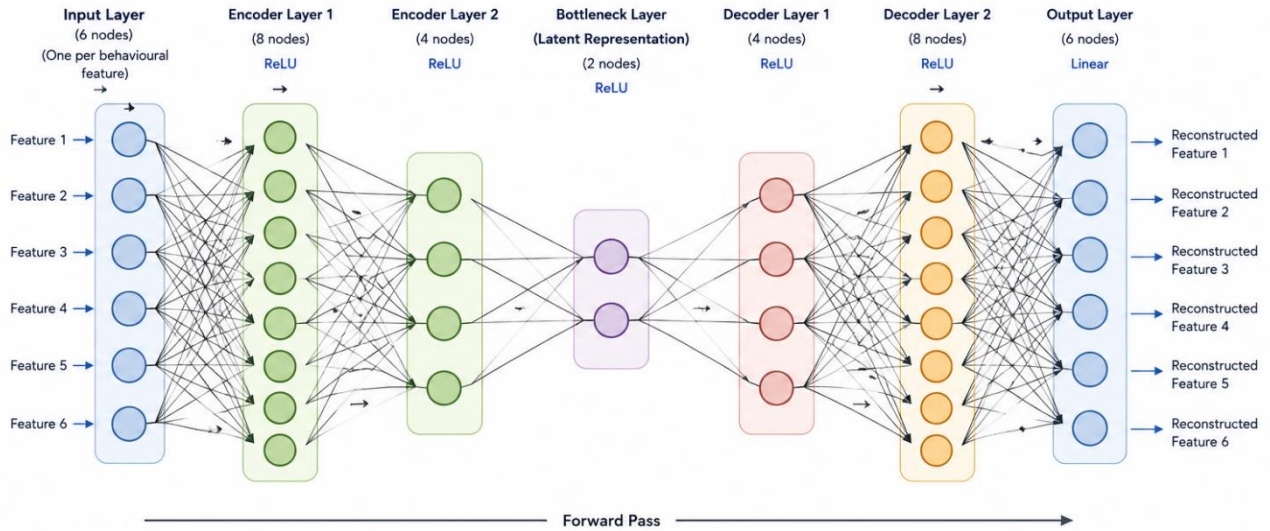


Figure 2.3: Autoencoder Architecture for Anomaly Detection

The ExamGuard autoencoder has a symmetric structure, such that the three layers of the encoder, with ReLU activation, correspond to the three layers of the decoder, with linear activation. The bottleneck layer of 2 neurons, compresses 6 features of the input into 2 dimensional representation. It is an architecture capable of having enough representational power for the behavioural data, and at the same time a useful compression ratio, so that the reconstructed data are distinguishable by the reconstruction error, in order to identify any anomaly. Only normal behaviour data was used for training (1000 samples) using the Adam optimiser and mean squared error loss function with early stopping to avoid overfitting the model.

A threshold for the anomaly was derived from the 95th percentile of the errors in the reconstruction of the normal training data, thereby ensuring that 95% of the normal data is reconstructed correctly or classified as normal, and that the data samples with anomalies are reconstructed incorrectly. Vincent et al. (2008) showed that AE with a bottleneck architecture successfully extracts informative representations of the predominant modes in the training data, and therefore performs well on semi-supervised detection of anomalies. The wide range of their applications, such as network intrusion detection (Javaid et al., 2016), medical anomaly detection (Baur et al., 2019) and financial fraud detection (Schreyer et al., 2017), suggests their potential applicability to examination behaviour monitoring.

2.6 Big Data in E-Learning Systems

E-learning platforms capture a lot of data about student interactions, for instance, how many pages are seen, logins are attempted, assignments are submitted, quizzes are taken, and forums are interacted with. Such platforms can serve as rich and detailed sources of behavioral information because they are able to capture student actions in fine-grained, real-time detail, Khan and Sharma (2020) propose. This phenomenon has been in line with the new paradigm of learning environments that generate data, characterized by the classical “3Vs” of Big Data: Volume, Velocity, Variety. With the increasing implementation of digital learning, comprehending and analyzing these massive data sets becomes crucial for enhancing security, performance metrics, and decision-making.

i. Volume

Every e-learning site processes millions of records every day, such as clickstreams, assessments, interaction histories, and multimedia usage. For example, in a single week of coursework, a single student can produce thousands of event logs (Papamitsiou & Economides, 2014). In a university environment, with thousands of students, these logs grow very large, up to terabytes.

ii. Velocity

Student interactions take place in real-time and generate data in real-time. For system events like login checks, access to the content or live assessments, they need to be processed continuously, particularly during exams or synchronous learning sessions.

iii. Variety

E-learning data can be structured or semi-structured (such as clickstreams, which are recorded as JSON), and unstructured (such as discussion text, uploaded files, video engagement logs). Big data analytics is thus an integral part of contemporary educational technology that will foster innovation in digital learning and enhance institutional decisions (Ifenthaler & Yau, 2020).

But, big data analytics is not always sufficient to detect anomalies like cheating patterns, bot activity or suspicious user activity and this is the reason the need for this topic. The core to this topic is by use of big data theory, which is scalable, has a high dimensional analysis (it can analyze many behavioral features at once) and can detect anomalies in real-time.

2.6.1 Big Data Theory (3Vs, 5Vs, 7Vs Models)

The Big Data Theory was first put forward by Laney (2001) and includes the characteristics of volume, velocity and variety. More recent researchers added Veracity, Value and Variability (Demchenko et al, 2014). These properties are all inherent in e-learning datasets, as they accumulate steadily and continuously as users interact with them, and the diversity of the types of interaction, as well as inconsistencies in human behavior, further increase these properties. As stated in Big Data Theory by Chen and Zhang (2014), large-scale analytics in education is a fitting use case for a distributed framework like Apache Spark or Hadoop. The 3Vs and 5Vs and 7Vs frameworks for Big Data Theory form the foundation of research that deals with large-scale digital datasets (Abbas et al., 2024). These models offer a framework for comprehending the dimensions, velocity, volume, reliability and analytic worth of big data systems. For e-learning platforms, these concepts illustrate the potential complexity of collecting, saving, analyzing and understanding large volumes of student behavior data. They also motivate the importance of scalable data architectures as well as powerful machine learning methods for anomaly detection, providing a direct theoretical motivation for your research project.

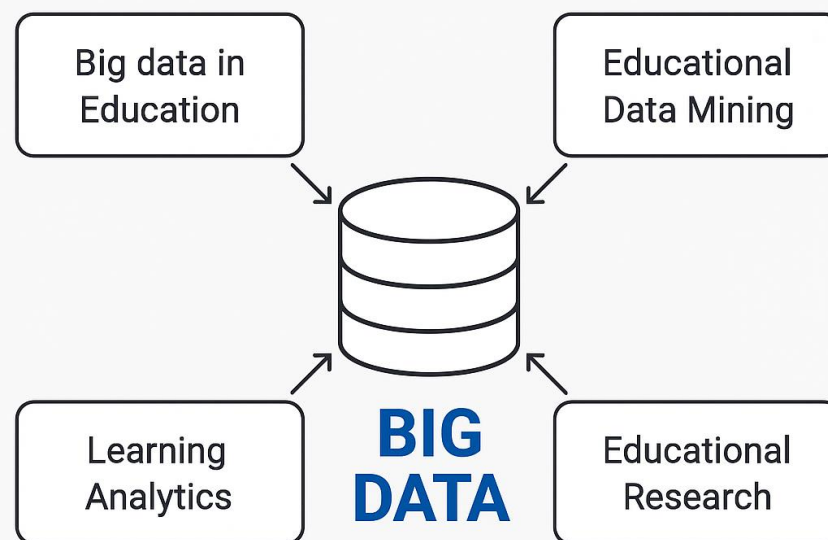


Figure 2.4: Importance of Big Data in E-learning

2.6.2 Outlier Theory

Outlier Theory presents statistical methods which classify anomalies as points that fall far from a predicted distribution. This theory gives rise to classical statistical techniques: the Z-score, Mahala Nobis distance, and density based techniques (Barnett & Lewis, 1994). In an e-learning context, this theory can be used to identify student behaviors that are not typical, and do not conform to normal distribution patterns, like study time that is very long or very short, or very long or very short quiz times, or very long or very short log-in time.

One of the earliest and most commonly used definitions of outlier was described by Hawkins (1980) as an "observation that deviates so much from the other observations as to arouse suspicion that it was generated by a different mechanism. The above definition reinforces the concept behind outlier analysis: Anomalies are usually the result of processes or behavior that is not representative of the normal state of the system.

2.6.3 Behavioural Analytics Theory

Behavioral Analytics Theory is a concept centered around the idea that human behavior can be quantified, modeled and interpreted using data-driven approaches. Siemens and Long (2011) believe that using behavior data in education can help institutions determine engagement, predict performance, and detect abnormal learning patterns. This theory is applicable to this study as there are anomalies that may find expression in behavioral deviations, which need to be studied systemically. This theory offers a solid lens to view the interactions between students and e-learning systems, as well as any variations in their behavior patterns, which can signal anomalies.

This theory is grounded in cognitive, psychological and computational approaches and provides a structured modelling, interpretation and evaluation framework for the development of machine-learning based anomaly detection systems for user behavior at scale. Behavioral Analytics Theory in this topic will facilitate the ability to infer patterns from huge logs of learning behaviours and deliver the best possible support for the detection of irregular learning, ensuring digital learning environments remain intact.

There are several important connections between Behavioral Analytics Theory and your work on Big Data Anomaly Detection Pipeline in E-Learning Systems:

- i. Describing "Normal" Student Behaviour
Behavioral analytics is the theory behind recognizing typical learning behaviour. The behavioral theory can be utilized to conceptualize the need for a learned baseline of normal activity for unsupervised machine learning algorithms.
- ii. Identifying abnormal or unusual patterns
Cheating, bot activity, impersonation, disengagement, etc., are all considered anomalies when they disrupt the behavior of a normal user. Theory of behavioral analytics can provide an explanation of the cause and possibility of measuring these deviations.
- iii. Ensuring support for big data analytics
There are lots of behavioral logs (clickstreams, timestamps, actions) created by e-learning platforms. The theory of behavioral analytics enables these massive volumes of data to be turned into readable behavioral models that can be used in the Big Data pipelines.
- iv. To improve the accuracy of the models of anomalies
Theoretical knowledge of human routines, cognitive patterns, and the contextual influences creates more accurate anomaly detection models, and reduces the number of false alarms.
- v. The use of linking behavior to learning outcomes
Behavioral analytics reinforces that anomalies aren't always technical glitches, but could be academic issues that include poor performance, abuse of the system or tampering of student accounts.

2.6.4 Feature Engineering for Examination Behaviour

Choosing and designing features for anomaly detection can greatly affect the system's performance on detecting suspicious behavior (Yamasari et al., 2024). The ExamGuard system utilizes six behavioral features which each capture a different aspect of the behavior of the student in the examination.

The examination score reflects the student's performance in the examination. Total score is not a definitive mark of cheating (high scores can be earned through good preparation), however it, along with other characteristics, may be a sign of cheating when the score is very high and the behaviour is suggestive of cheating. Total clicks indicate the intensity of the mouse activity during the exam session; if the number of clicks is unusual, it could mean that the user might be trying to access other resources.

Time on exam is a measure of overall time spent on the examination. Completion times that are brief compared to the time limits of the examination may be a result of using prepared responses whereas longer times may be a result of using external materials to a greater extent than usual. A tab switch occurs when a student moves away from the examination window, which represents a crucial aspect of online cheating. Average time per question is a normalised measure of engagement with the question, with very short average times may reflect speedy and confident answers that are not commensurate with the difficulty of the exam.

Activity rate is computed as total clicks / total time on exam, reflecting the intensity of interaction. Rates that are significantly higher than normal can reflect frenzied search behavior and rates significantly lower than normal can suggest little interest in the examination interface. The joint effect of the six features gives a multidimensional description of the examination behaviour which is more discriminating than any of the features alone..

Table 2.3: Behavioural Features Used for Anomaly Detection

Feature Name	Description	Expected Normal Range
Score	Percentage score achieved	40-80%
Total Clicks	Total mouse clicks during exam	30-150
Time on exam	Total seconds on exam	600s-1800s
Tab Switches	number of tab/window switches	0-3
Avg Time per Question	seconds per question on average	120-480s
Activity Rate	clicks per second	0.03-0.15

2.6.5 Comparative Analysis of Isolation Forest and Autoencoder

Isolation Forest and Autoencoder are two very different methods for anomaly detection, with their own unique pros and cons when applied to the context of examining behaviour in a science lab. Isolation Forest is an ensemble tree based method that is free of any distributional assumptions and works well in high dimensional spaces. It is very efficient in training and inference, which makes it suitable for real-time deployment (Chen & Zhang, 2024). But when it comes to anomalies, Isolation Forest may have trouble when it means something that is not an outlier in the feature space, but rather a combination of normal feature values.

As a neural network, the Autoencoder could learn more complicated non-linear relations between features and could identify anomalies that are not easily identified using linear partitioning techniques. It is more resource-intensive and hyperparameter sensitive to train, but has low computational cost for inference. A potential disadvantage of the Autoencoder is its semi-supervised learning, which is typically exposed to only normal data and may be more sensitive to subtle anomalies preserving some of the characteristics of the normal feature but with unusual patterns.

In reality, the two algorithms might detect different sets of the anomalous instances. Anomalies that are flagged by both algorithms are considered high confidence anomalies and anomalies flagged by only one algorithm are candidates for further investigation. ExamGuard's comparative evaluation framework gives the examiner the empirical basis to decide which algorithm is best suited for their institutional environment, with respect to the false-positive failure rate, and the cost of not detecting authentic anomalies.

2.6.6 Empirical Studies on Big Data in Educational Systems

It has been proved through several empirical studies that Big Data analytics plays an important role in enhancing the educational outcomes and operational decision-making. Lee, Cheung and Chen (2021) concluded that LIDs can give actionable insights that enable institutions to tailor interventions at a large scale. Thus, Ifenthaler and Yau (2020) found that big data analytics increases early detection of learning difficulties. The results demonstrate that educational data mining is an important part of the modern digital learning ecosystem.

The use of learning management systems (LMS), digital assessment tools and real-time tracking of students' activities has grown in the last decade and is actively being used in educational environments, leading to an ever increasing body of empirical research on big data in education. In summary, these studies highlight the promise of big data methods such as learning analytics, machine learning, and large-scale data processing pipelines in enhancing educational results, identifying atypical behaviors, and enabling data-informed decisions. The collection of pedagogically oriented empirical research gives a concrete base to the anomaly detection systems in e-learning platforms, as it shows the trends and limitations of the methodologies.

2.6.7 Empirical Studies on Anomaly Detection in E-Learning

Studies focusing on anomaly detection in learning environments are still emerging (empirical). Previous studies have investigated how to detect anomalous behaviors on online learning platforms using anomaly detection methods. For example, Qiu et al. (2021) used clustering-based approaches to

identify cheating on online exams and showed that the approaches could significantly enhance the detection of cheating by trying to complete the quiz quickly, or with suspicious behavior. The results from the other study showed that machine learning models were much more successful in detecting anomalies in student logs compared to rule-based systems, as demonstrated by the isolation-based techniques used by Alzahrani (2021). These research works highlight the importance of anomaly detection in ensuring academic honesty.

Unsupervised deep learning using autoencoders for identifying rare engagement patterns of students in large blended learning courses. Their findings highlight the significance of unsupervised learning when anomalies are limited or not available. (Kim, Shaw & Lee ,2021)

Implement Isolation Forest and Local Outlier Factor (LOF) with MOOC datasets to detect suspicious activity including multiple rapid quiz attempts, account sharing and bot-like interactions. They confirmed their empirical results with the appropriateness of the Isolation Forest anomaly detection method for e-learning (Hellenbrand & Stadler, 2020).

Methods of outlier analysis to identify unusual behaviours of students in LMS environments, including unusual submission patterns and sudden inactivity. The study revealed that there are anomalies which have a correlation with academic risk and dishonesty (Akçapınar, 2019).

2.6.8 Empirical Use of Machine Learning Models for Anomaly Identification

In anomaly detection tasks, ML models like Autoencoders and One-Class SVMs are found to be effective in high dimensional tasks. Wang et al. (2020) proved that the Autoencoder achieves good detection accuracy when using the user behavior data of digital systems. Also, Liu et al. (2012) proved the effectiveness of Isolation Forests to find rare and subtle anomalies in large datasets. The results are consistent with the proposed use of the ML based method as a means of detecting irregular student activities.

There is a proliferation of empirical research using large education datasets to predict educational outcomes that employ machine learning. Baker and Yacef (2009) showed that ML models can detect student confusion, gaming-the-system behaviors and off-task activities in real time, based on thousands of logged interactions. Their research also empirically demonstrates that there are detectable digital footprints of anomalous educational behaviors.

Furthermore, Márquez-Vera, Cano, and Romero (2013) analyzed more than 18,000 high school students records and using data mining and machine learning methods, they obtained excellent

predictions levels in academic risk. Their empirical results confirm the trustworthiness of ML models at detecting irregular patterns in large education data sets.

2.6.9 Empirical Studies on Data Pipelines for Real-Time Analytics

Across industries, real-time data processing pipelines have become quite popular. Zaharia et al. (2016) showed that Apache Spark Streaming can be used to efficiently process realtime log data for high-velocity. Paredes and Martinez (2019) reported that in an educational setting streaming pipelines can dramatically increase the responsiveness when monitoring students' behaviour. But most of the studies are carried out for prediction or personalization and not for anomaly detection, which is what this study aims to do.

2.7 Review of Existing Systems

In this section we would be reviewing the already existing scheduling systems.

2.7.1 Commercial Online Proctoring Solutions

A number of commercial solutions have been developed to overcome the problem of the integrity of online examinations. The major commercial alternatives are ProctorU, Examity and HonorLock, which rely primarily on live human proctors and automated detection systems. Typically, students will need to install browser extensions or dedicated applications, allow access to webcam and microphone streams, and agree to being recorded and stored during the exam. Commercial proctoring services can provide full monitoring, but can have a number of drawbacks in educational environments. Privacy is a primary concern with nearly 80% of students saying that they were uncomfortable with being proctored via a webcam (Swauger, 2023); critics have pointed out that webcam proctoring discriminates against certain demographic groups (Swauger, 2023). Many institutions, especially in developing areas, cannot afford the cost of commercial proctoring services. Students also in resource-poor settings are further removed because of technical requirements such as access to reliable and high-speed internet connections and modern computing equipment.

2.7.2. Learning Management System Integrity Features

Some major LMS platforms offer basic integrity checks like question randomisation, time limits and browser lock plugins. Some major learning management systems offer basic integrity checks like question randomisation, time limits and browser lock plugins. The integration of the Safe Exam Browser in Moodle allows students to only view and use Moodle while taking an exam and prevents the student from using any other app. Blackboard Integrity offers a few monitoring options and does

not offer sophisticated anomaly detection. Canvas supports third-party proctoring services, and has no built-in behavioural analytics. One limitation of these platforms is that they can only be used to control behavior and not to detect it. If a student manages to bypass restrictions, he or she will not be flagged, and if a legitimate student has technical problems, he or she can be incorrectly flagged. Behavioural monitoring methods, on the other hand, identify irregular activities even if limit policy is circumvented, complementing integrity assurance.

2.8 Summary

The literature was reviewed about e-learning, academic integrity, anomaly detection, and the machine learning algorithms used in the ExamGuard system. The review finds a considerable mismatch between the development of Online Exam Delivery and the access to accessible, privacy protecting integrity monitoring solutions. There are commercial proctoring options available, but they may not be ideal for a variety of educational settings because of privacy concerns, expense, and technical demands.

Isolation Forest and Autoencoder algorithms are two well-known and theoretically sound algorithms that have been proven to work well in many application areas for unsupervised anomaly detection. They are novel in the context of e-learning examination behaviour monitoring and tackle the challenge of detecting anomalous behaviour without the need of labelled training data. Developed as part of this project, the ExamGuard system is a viable solution to the academic integrity challenge in online teaching and learning by combining both algorithms within the framework of a web-based e-learning platform that also supports an integrated comparative evaluation framework.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.1. Preamble

This chapter provides the analytical and design of the ExamGuard system. It starts by analysing both functional and non-functional requirements, then explains the architecture and physical design of the system, using UML diagrams to illustrate the logical design, and finishes with the evaluation framework used to evaluate the system's performance and usability.

3.2. Requirement Analysis

Requirement analysis is the process of determining and describing the requirements of the system required for the successful creation and use of the system. The need analysis of this project is split into:

1. Functional Requirements
2. Non-Functional Requirements

3.2.1. Functional Requirements

The functional requirements are a description of the behaviours and functions that ExamGuard system needs to support. The following requirements were identified based on the use cases for the system as well as the needs of the two main groups of users, students and examiners.

Table 3.1: Functional Requirements of ExamGuard

S/N	Requirement Category	Description
1	User Authentication	System shall support registration and login for both student and examiner roles
2	Exam Management	Examiners shall be able to create, edit, and delete examinations and questions
3	Exam Delivery	System shall deliver examinations to students and enforce time limits

4	Behaviour Monitoring	System shall track and record tab switches, clicks, and time during exam
5	Anomaly Detection	System shall apply both IF and AE models to submitted behaviour data
6	Results Management	System shall store and display examination scores and anomaly results
7	Examiner Dashboard	Examiners shall view live students, behaviour logs, and anomaly flags
8	Algorithm Evaluation	System shall display comparative performance metrics for both algorithms.

3.2.2. Non-Functional Requirements

Non-functional requirements include quality attributes and constraints that the ExamGuard system needs to meet in addition to its functional requirements. These requirements guarantee that the system is not only correct in its functionality but usable, reliable and meaningful in the real educational environment.

Table 3.2: Non-Functional Requirements of ExamGuard

S/N	Requirement	Description
1	Performance	System shall respond to user interactions within 2 seconds under normal load
2	Scalability	System shall support concurrent examination sessions by multiple students

3	Security	All user passwords shall be hashed; session data shall be protected
4	Usability	System shall be operable by users with basic digital l
5	Reliability	Anomaly detection shall execute for every examination submission
6	Privacy	System shall not collect audio, video, or biometric data from students
7	Portability	System shall operate on standard web browsers without additional plugins

3.3. System Architecture

The ExamGuard system is designed in 3 layer web architecture, which includes presentation layer, application layer, and data layer. This structure has a good separation of concerns which helps in maintaining, testing and extension of the system in future.

The presentation layer includes the web-based front ends that are available for student and examiner users via any standard web browser. The student facing interfaces consist of log-in interface, examination interface and results interface. Examiner facing interfaces consist of login page, examination management interface, algorithm evaluation dashboard and an examiner dashboard which has real-time student monitoring. The evaluation dashboard is implemented with HTML5, CSS3, JavaScript and data visualisation is achieved with Chart.js.

Flask Web Framework, in Python, is used to implement the application layer. Flask is used to route HTTP requests, handle user sessions, execute business logic, and provision API endpoints. In the application layer, the anomaly detection module is responsible for loading the trained machine learning models and calculating an anomaly score for each examination session that is submitted. The

behaviour tracking module processes the realtime behaviour data that comes from the frontend. Authentication module handles user registration, login and session validation.

The data layer is a SQLite relational database which is handled by SQLAlchemy, the ORM for Flask. Accounts, examination definitions, questions, examination sessions and behaviour logs are stored in the database. The machine learning models (trained Isolation Forest, Autoencoder, and StandardScaler) are stored as serialised files with joblib (Isolation Forest and StandardScaler) and

TensorFlow's native format (Autoencoder), and loaded into application memory on start-up to reduce inference latency.

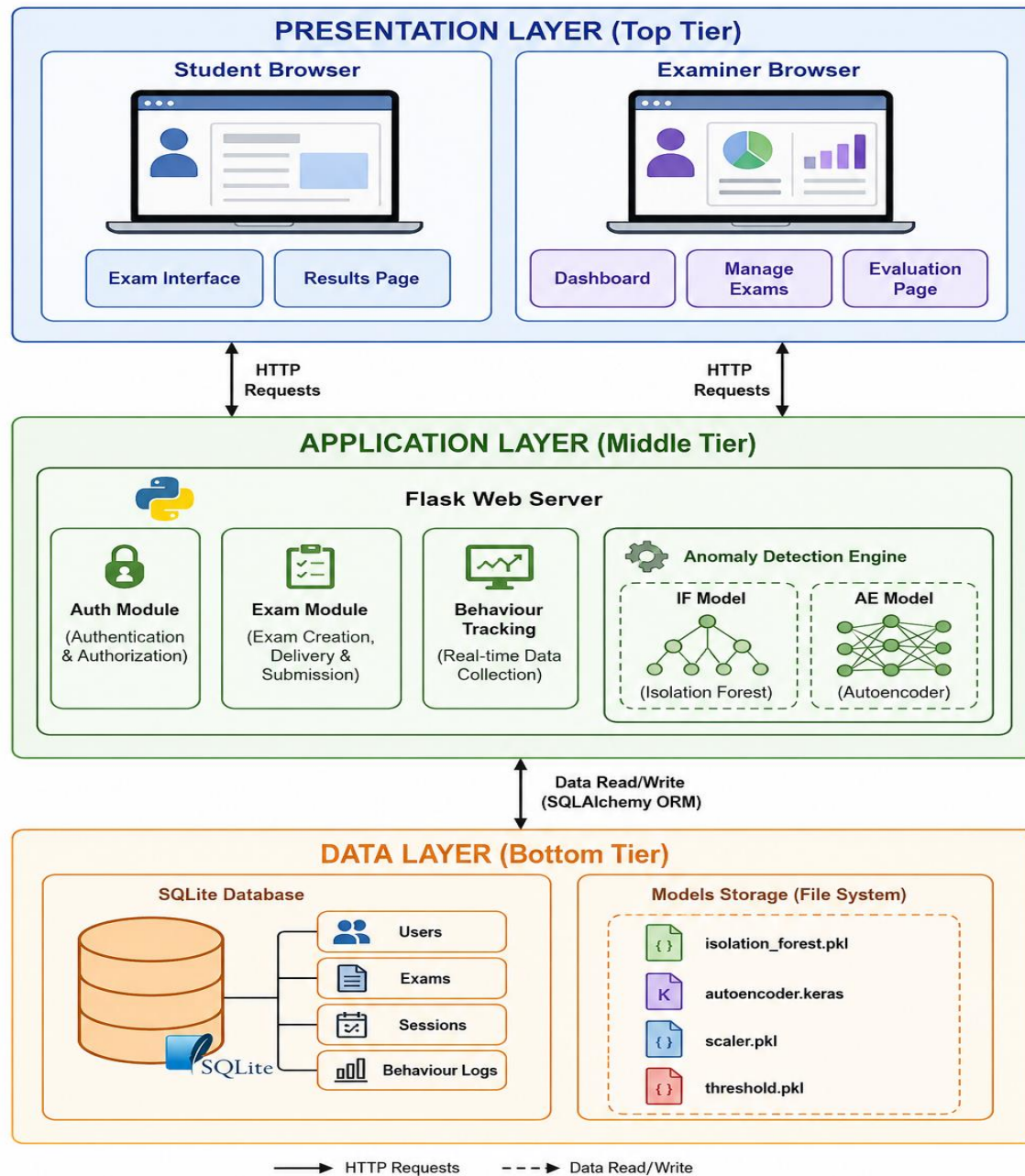


Figure 3.1: System Architecture

3.4 Physical Design

The conceptual design is a part of the logical design of the system, which determines the system architecture, data model, infrastructures and algorithms. Logical design is concerned with the development of high level presentation of the system's form and behavior but not with the physical detail of the implementation. The purpose of the logical design is to establish a definitive understanding of the way the detection system will run and interact with the user and other systems.

3.4.1 Machine Learning Pipeline

The steps in the machine learning pipeline for ExamGuard are: data generation, preprocessing, model training, and deployment. As a result of the ethical and practical limitations of collecting real-world examination data labelled for both models, a synthetic behavioural data set was created to train both sets of models. The behavioural trait of normal and abnormal students as described in the literature was used for informing the synthetic data generation process.

Normal student behaviour was simulated by Gaussian distributions for each feature with parameters based on plausible examination performance: scores normally distributed around 65 (SD 15), total clicks normally distributed around 80 (SD 20), time on exam normally distributed around 1,200 seconds (SD 300), tab switches normally distributed around 1 (SD 1), average time per question normally distributed around 240 seconds (SD 60), and activity rate normally distributed around 0.07 (SD 0.02). Higher scores (mean 85) and more clicks (mean 200), shorter examination times (mean 300 seconds), more tab switches (mean 10), shorter per question times (mean 60 seconds), and higher activity rates (mean 0.5) were modelled as anomalous behaviour which represented potential cheating patterns.

Combined data set (1100 samples) was used to fit the StandardScaler and then all the features were normalized to zero mean and unit variance prior to training the model. The scaled combined dataset was used to train the Isolation Forest, and the Autoencoder was trained only with the 1,000 normal samples to get a model of normal behaviour. The trained models and the scaler were saved to disk and loaded when the app started.

3.4.2 Behaviour Tracking Module

The behaviour tracking module works at the front end, with the ability to listen to student interactions with the examination interface in real time using JavaScript event listeners. Tab switch events are detected in two ways: Through the Page Visibility API and the window.onblur event, and are counted each time the student moves from the examination window. A document level click event listener is

used to track total clicks. The time on exam is calculated as the time elapsed from the start of the exam to its end, the duration of which will be shown on the timer to the student.

They are behavioural metrics that are gathered during the examination session, and sent to the Flask back-end as part of the examination submission payload, in JavaScript variables. The pipeline of derived features (average time per question, activity rate) is computed in the backend and all metrics are logged in the BehaviourLog database table before calling the anomaly detection pipeline.

3.5 Logical Design

The conceptual model is part of the logical design of the system which defines the systems architecture, data model, infrastructures and algorithms. Logical design is concerned with the development of high level presentation of the system's form and behavior but not with the physical detail of the implementation. The purpose of the logical design is to establish a definitive understanding of the way the detection system will run and interact with the user and other systems.

3.5.1 Sequence Diagram

A sequence diagram is a form of Unified Modelling Language (UML) diagram that shows how objects interact in a particular scenario by showing the messages sent between the objects as they pass through time. It shows the flow of messages and information, helping us to understand the dynamic behavior of a system.

UML Sequence Diagram – AI-Powered Exam & Anomaly Detection

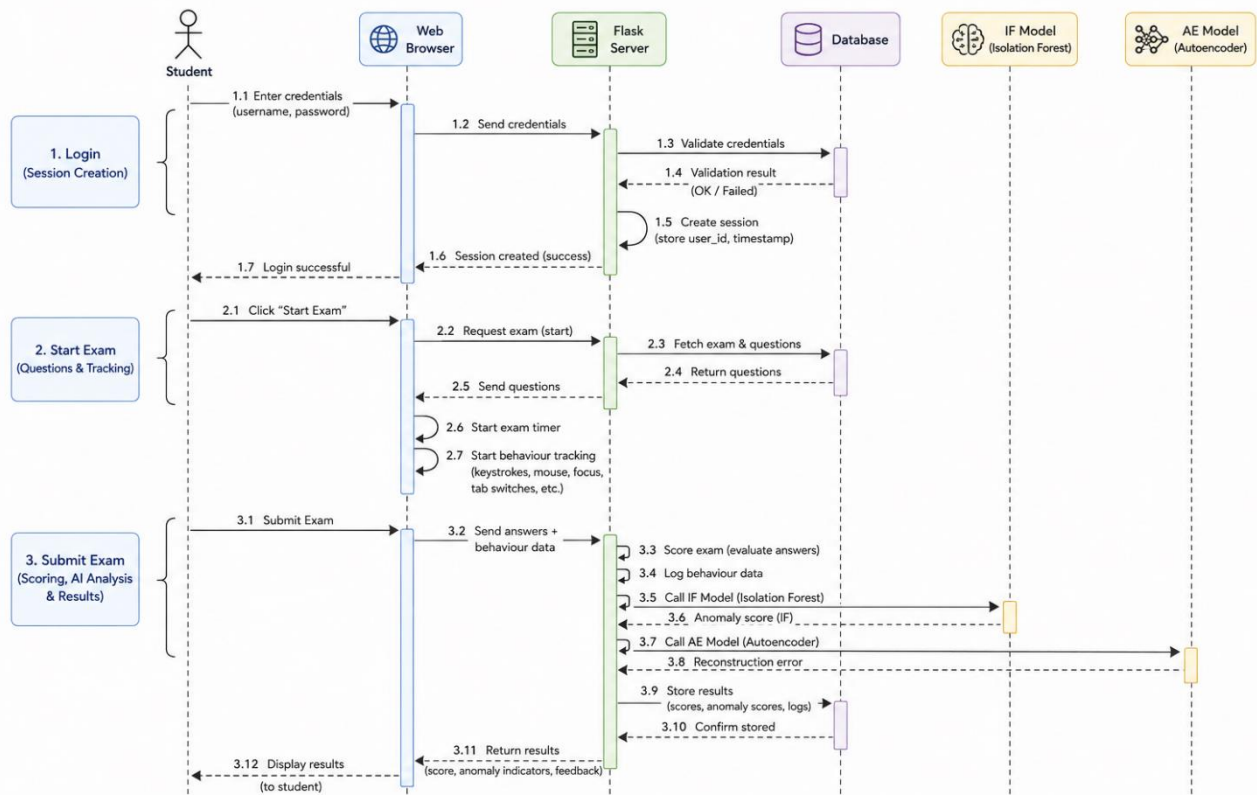


Figure 3.2: Sequence Diagram of the ExamGuard System

3.5.2 Class Diagram

A UML diagram that shows the structure of the system by its classes, their methods and attributes and the relationship between the classes is called a class diagram. A class diagram gives a system's structural perspective and allows the design and the organization of the classes of the system to be envisioned. The system's class diagram has various classes that interact with each other to facilitate anomaly detection as depicted in the figure below

UML Class Diagram – SQLAlchemy Models

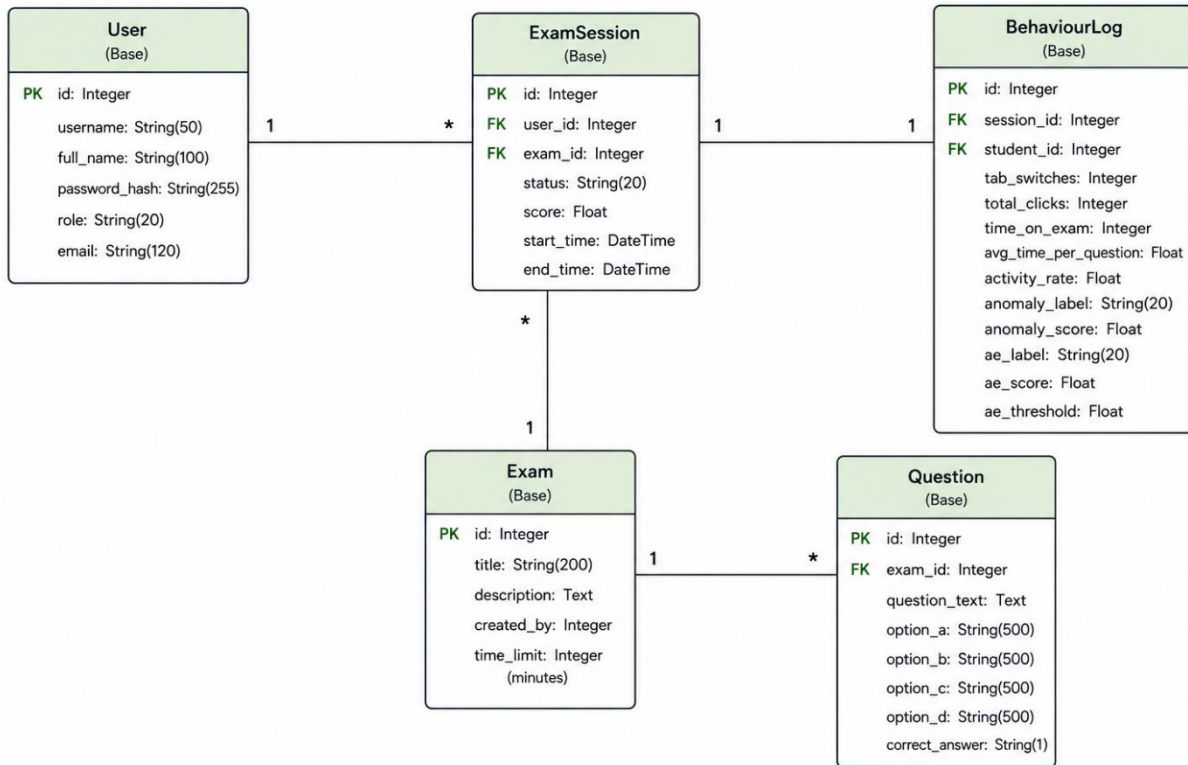


Figure 3.3: Class Diagram of the ExamGuard System

3.5.3 Use-Case Diagram

A use case diagram is a kind of UML behaviour diagram portraying the interaction among actors (users as well as external systems) and a system with the purpose of reaching certain goals. It is useful for comprehending the user perspective of the system.

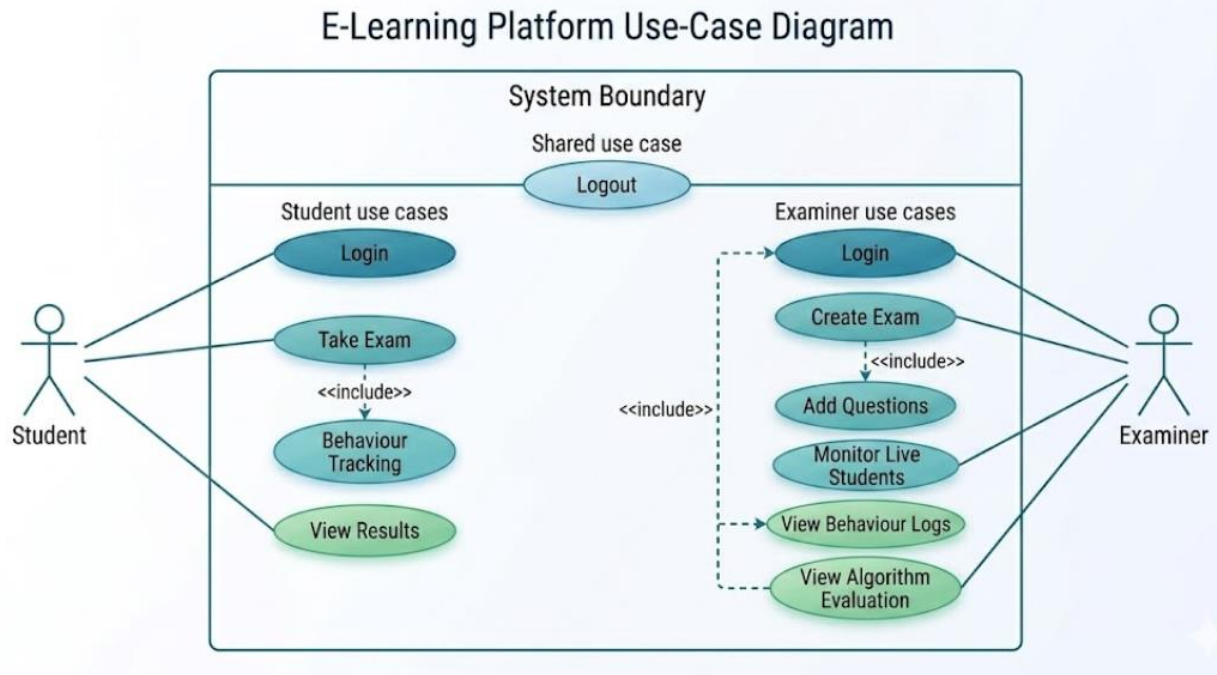


Figure 3.4: Use-Case Diagram of the ExamGuard System

3.5.4 Activity Diagram

An activity diagram is a UML behavioural diagram which depicts a flow of activities and actions in a system. It depicts the order in which operations are to be performed, the decision points, and the direction of control in a process from start to end.

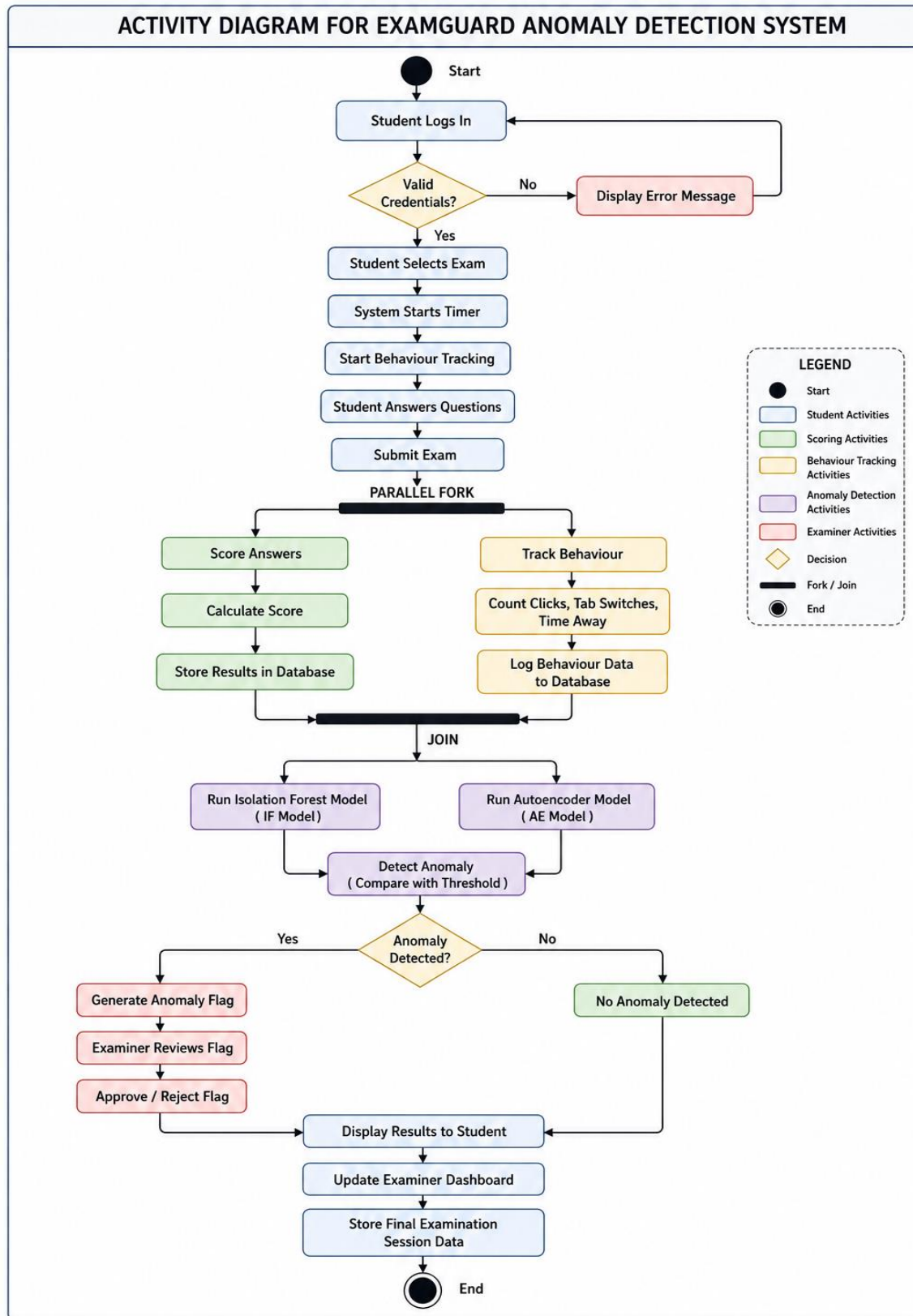


Figure 3.5: Activity Diagram of the ExamGuard System

3.5.5 Entity Relationship Diagram

A conceptual depiction of a database's logical structure is called an entity-relationship diagram (ERD). It demonstrates relationships between objects or concepts (items or ideas) and their attributes. The ERD contains entities like User, Exam, Exam_Session, Question, Behaviour_Log along with relationships illustrating their connections, in the context of the anomaly detection system.

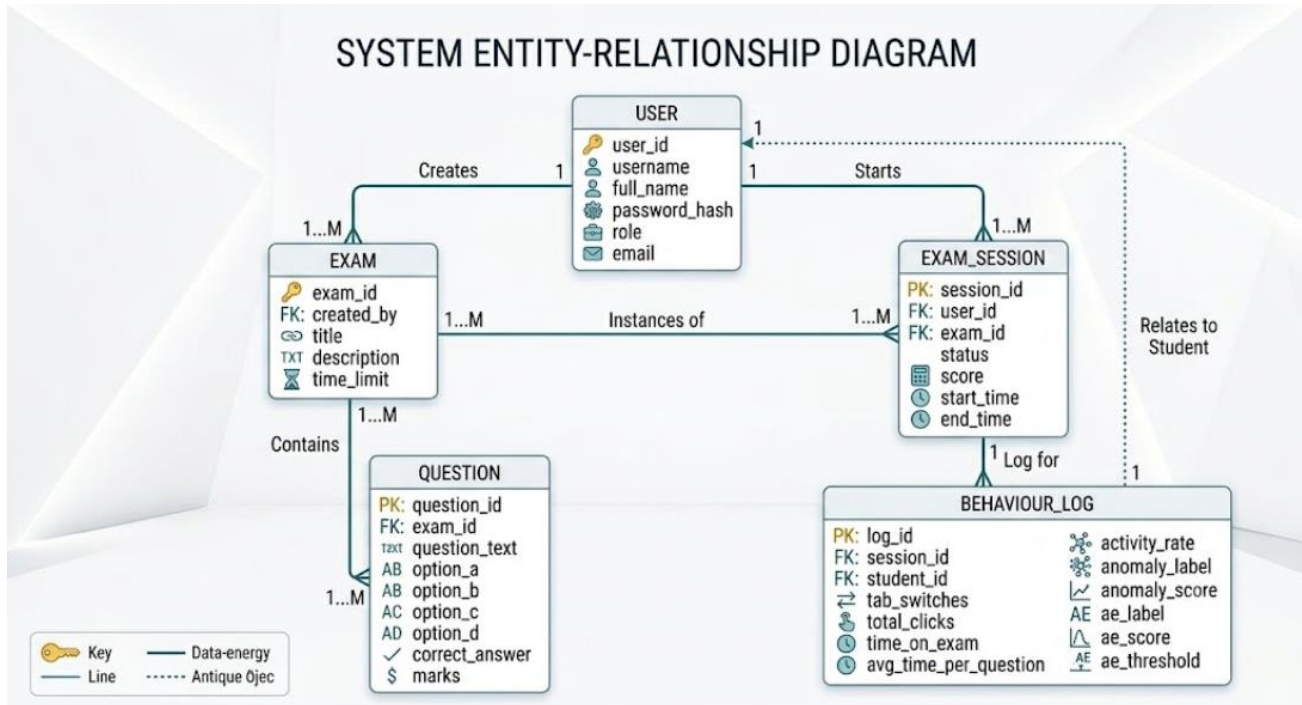


Figure 3.6: Entity Relationship Diagram of ExamGuard

3.6 Evaluation of the System

Evaluation is an extremely important step in the lifecycle of any system. The following describes the methodology used in a systematic manner to assess the performance and capabilities of the automated meeting scheduling system. This evaluation will also confirm that the system operates as described and is reliable and correct, and measure the performance of the system, it's usability, user satisfaction and experience of using the system.

3.6.1 Algorithm Evaluation Criteria

The comparative evaluation of the Isolation Forest and Autoencoder algorithms is done using five quantitative metrics computed on the synthetic dataset, which has a ground truth class label (normal versus anomalous) with which to compare the predictions of the algorithms. Precision is the percentage of cases identified as such by the algorithm that are actually anomalous. The higher the precision, the more likely the algorithm will be correct when it generates an alert. High precision in the examination environment reduces the likelihood of erroneous accusations of cheating by students who are actually not. Recall is the percentage of "abnormal" instances correctly recognized by the algorithm. High recall means that cases of actual misconduct are not overlooked. The trade-off between precision and recall is a basic factor that needs to be taken into account when choosing an algorithm as the importance of each will be determined by the institutional policy of the false positive/false negative cost. F1 Score gives a harmonic mean of precision and recall, which gives a single number that takes into account both. It is also helpful if the class distribution is skewed, in this application, there are significantly fewer anomalous instances than normal instances. The Area Under the ROC Curve (AUC) quantifies the algorithm's discrimination power over all possible classification thresholds. An AUC of 1.0 means that it discriminates perfectly, a score of 0.5 means the model is performing at chance level. The Silhouette Score is a measure of the quality of clustering obtained from each algorithm binary prediction, indicating the distance between the normal and abnormal clusters in the feature space.

3.6.2 Statistical Analysis Method

Each of the questionnaire items is described with descriptive statistics such as mean and standard deviation to define the central tendency and variability of the ratings of user satisfaction. This enables dimension-level analysis with the mean scores computed separately for each dimension of the PSSUQ (System Usefulness, Information Quality, Interface Quality, and Overall Satisfaction). As in the original PSSUQ, lower mean scores on the score sheet (1-7 scale) mean higher levels of satisfaction. The algorithm evaluation metrics are displayed numerically in a comparison table and graphically using various types of charts such as a grouped bar chart, ROC curve overlay, score distribution histograms, scatter plot, radar chart, and confusion matrices. It is a multi modal presentation which helps inform examiners to make informed judgements on the suitability of algorithms for their institutional context.

CHAPTER FOUR

SYSTEM IMPLEMENTATION AND EVALUATION

4.1 Preamble

This chapter gives an overview of the realization of an e-learning environment equipped with the ExamGuard anomaly detection system. It outlines the system requirements, implementation tools, system development methodology, system evaluation outcomes, and important program modules and interfaces. The chapter shows the translation of the design described in Chapter Three to a real implementation of the system via web-based implementation incorporating real-time behaviour monitoring and dual-algorithm anomaly detection.

4.2 System Requirements

The standards and features that a system has to have in order to meet the needs of its users are referred to as system requirements. These requirements are divided into two categories: Hardware requirements and software requirements.

4.2.1 Hardware Requirements

The hardware requirements specify the physical computing hardware that is required to implement the ExamGuard system. The minimum hardware requirements are shown in table 4.1.

Table 4.1: Hardware Requirements Table

S/N	Requirement	Specification(s)
1	Laptop/Desktop Size	11 inches or larger for adequate interface visibility
2	Processor	Intel Core i5 or equivalent or higher
3	Primary Memory	8 GB RAM or higher (4 GB minimum for model loading)
4	Architecture	64-bit (x64) for Windows / ARM x64 for macOS
5	Secondary Storage	512 GB SSD or higher (models require approximately 50 MB)
6	Display	1280 x 720 resolution or higher

7	Network	Stable internet connection for web application access
---	---------	---

4.2.2 Software Requirements

The software requirements define information about the operating system and software requirements for the correct operation of the ExamGuard system. The software needs are listed in Table 4.2.

Table 4.2: Software Requirements Table

S/N	Requirement Category	Specification
1	Operating System	-column table with headers: S/N, Requirement Category, Specification. Rows: (1) Operating System — Windows 10+, macOS 11+, Ubuntu 20.04+
2	Python Version	Python 3.10 or higher
3	Web Framework	Flask 2.0+
4	ML Libraries	scikit-learn 1.0+, TensorFlow 2.10+, NumPy 1.21+, pandas 1.3+
5	Database	SQLite 3 (via SQLAlchemy)
6	Frontend Libraries	Chart.js 4.4.1 (CDN), HTML5, CSS3, JavaScript ES6+
7	Development Tools	Visual Studio Code, Python pip package manager
8	Web Browser	Chrome, Firefox, Edge

4.3 Implementation Tools

The following tools and technologies used for the development and deployment of ExamGuard system:

4.3.1 Python

The backend app and machine learning pipeline are primarily written in Python. This is because it has a large ecosystem of scientific computing and web development libraries that are ideal for projects that involve data science and web application development.

4.3.2 Flask

A lightweight web framework for Python that offers the ExamGuard backend with its HTTP routing, request processing, session handling, and template rendering. It is extremely simple to control how it is structured, and is an excellent choice for projects of this scale.

4.3.3 SQLAlchemy

SQLAlchemy offers an Object-Relational Mapping (ORM) layer that hides away using the database, allowing you to perform database operations in Python, instead of plain SQL. This makes development easier and makes the system more portable to various database back-ends.

4.3.4 scikit-learn

The isolation forest implementation for use with the ExamGuard anomaly detection pipeline is taken from scikit-learn. It has a well documented API and extensive documentation which makes it the standard library for classical machine learning in Python.

4.3.5 TensorFlow and Keras

The Autoencoder model is built, trained and deployed using TensorFlow and its high-level API, Keras. The TensorFlow's model serialisation in .keras format helps to efficiently save and load models.

4.3.6 NumPy and pandas

Throughout the data generation and preprocessing, NumPy offers efficient numerical array operations. DataFrame structures are used in pandas for data manipulation for model training.

4.3.7 joblib

Joblib is an efficient way to persist and load the trained Isolation Forest model and StandardScaler that is used in the code.

4.3.8 Chart.js

The frontend evaluation dashboard uses the Chart.js JavaScript charting library to create interactive charts such as ROC curves, bar charts, scatter plots, radar charts and doughnut charts.

4.3.9 Visual Studio Code

Throughout the process, Visual Studio Code was used as the main code editor with syntax highlighting, debugging support, integrated terminal and version control integration.

4.4 Development Methodology

ExamGuard was developed in an Agile manner, with an iterative approach, frequent testing, and small increments in features. The development process was divided into 5 consecutive sprints, which provided each one with a functional piece of the system.

The initial sprint was dedicated to establishing the basic infrastructure, including the structure of the Flask application, database schema design and implementation with SQLAlchemy, and implementing

student and examiner user authentication systems (registration and login). The second sprint provided the basic examination functionality: creation of examinations, examination management for the examiner, examination delivery for the student and basic recording of results. The third sprint focused on implementing the behaviour monitoring and anomaly detection pipeline: JavaScript based behaviour tracking on the front-end, add pre-trained Isolation Forest and Autoencoder models to the Flask back-end, and automated anomaly scoring when exams are submitted. The fourth sprint focused on the development of the examiner monitoring interfaces, including the live student monitoring dashboard, behaviour logs view and examination management interfaces. The 5th sprint involved the introduction of the algorithm evaluation dashboard (including the /evaluation_data API endpoint that calculates all of the quantitative metrics and the interactive Chart.js visualisations).

The continuous testing was carried out during development, where individual components were successfully tested by unit tests and integration tests were used to test the end-to-end examination workflow. The user acceptance testing took place with a small group of potential system users to obtain the users' feedback on any usability problems prior to formal evaluation.

4.5 Evaluation of the System

This section outlines the major functional modules of ExamGuard system and interfaces where the user interacts with the system. The modules will collectively enable the delivery of examinations, tracking of behaviour, identification of anomalies, and evaluation of performance.

4.5.1 Algorithm Performance Evaluation Results

The quantitative assessment of the performance was performed on synthetic training dataset of 1000 normal and 100 abnormal samples. Ground truth labels provided by the data generation process enabled the computation of classification metrics for both algorithms. Five metrics have been computed: precision, recall, F1 Score, AUC and Silhouette Score.

Table 4.3: Algorithm Performance Metrics Comparison

Metric	Isolation Forest	Autoencoder
Precision	0.9091	0.6667
Recall	1	1

F1 Score	0.9524	0.8
ROC AUC	1	1
Silhouette Score	0.6591	0.568

The ROC curve for each algorithm was created by changing the classification threshold and calculating the corresponding true positive rate and false positive rate. The Area Under the ROC Curve is used to measure the overall discriminative ability, rather than at any specific threshold. The score distribution histograms for the algorithms show how well the algorithms are able to distinguish between the normal and anomalous scores, giving an intuitive understanding of the models' discriminative capabilities. The confusion matrices provide detailed information about the classification errors for both algorithms, presenting the number of true positives, true negatives, false positives, and false negatives. The scatter plot of Isolation Forest scores against Autoencoder reconstruction errors shows the spatial separation of the normal and anomalous instances in the two dimensional score space and the level of agreement between the two algorithms when colours are used to represent ground truth label.

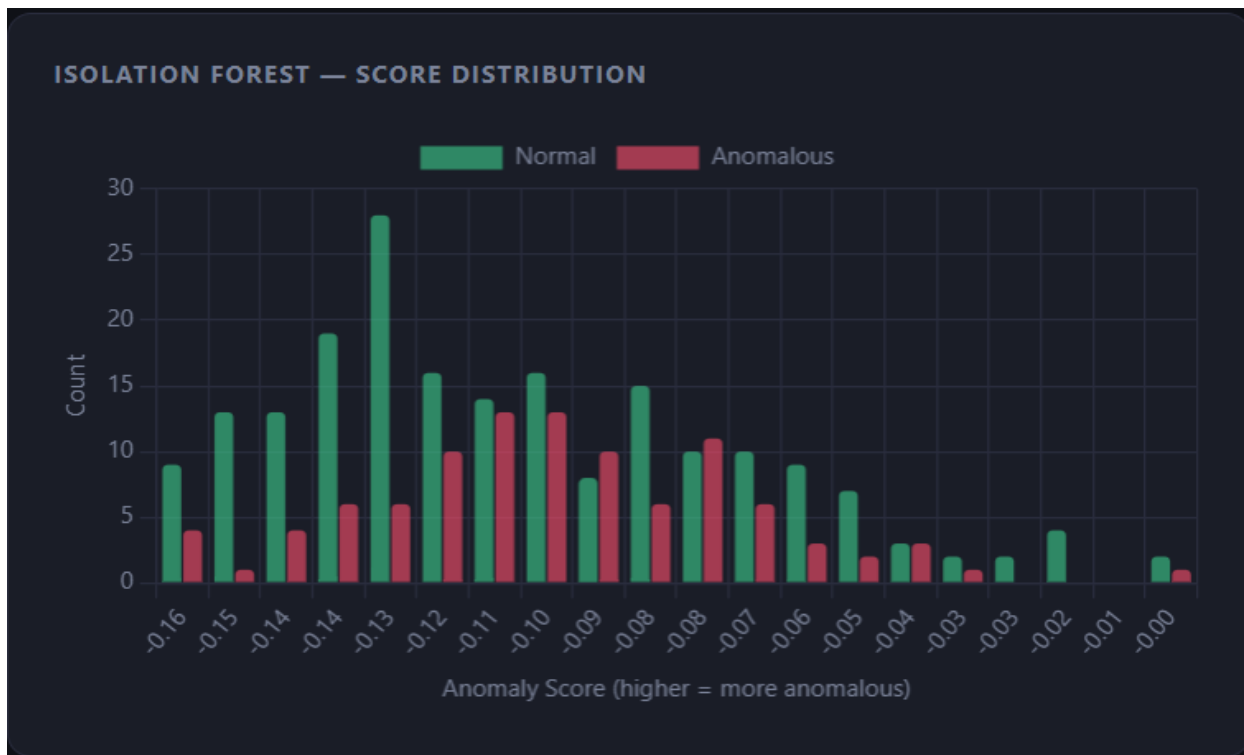


Figure 4.1: Isolation Forest Score Distribution Chart

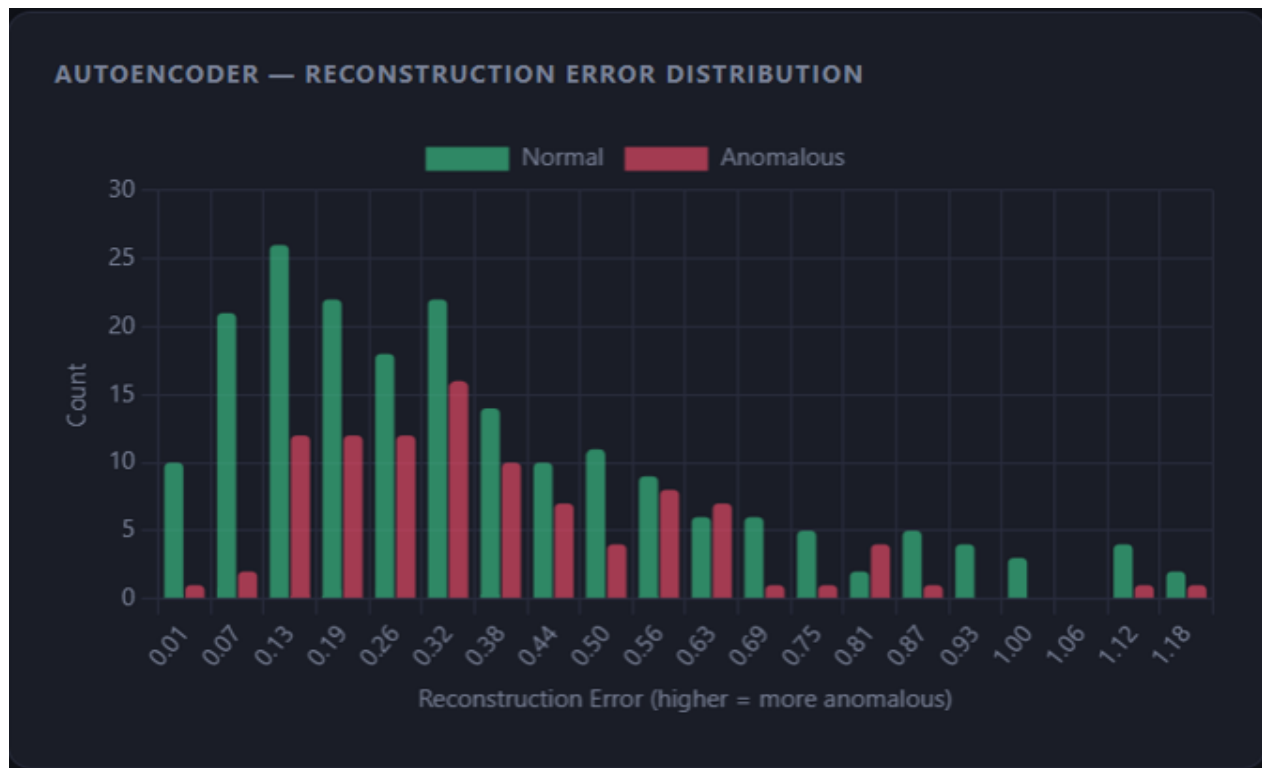


Figure 4.2: Autoencoder Reconstruction Error Distribution Chart

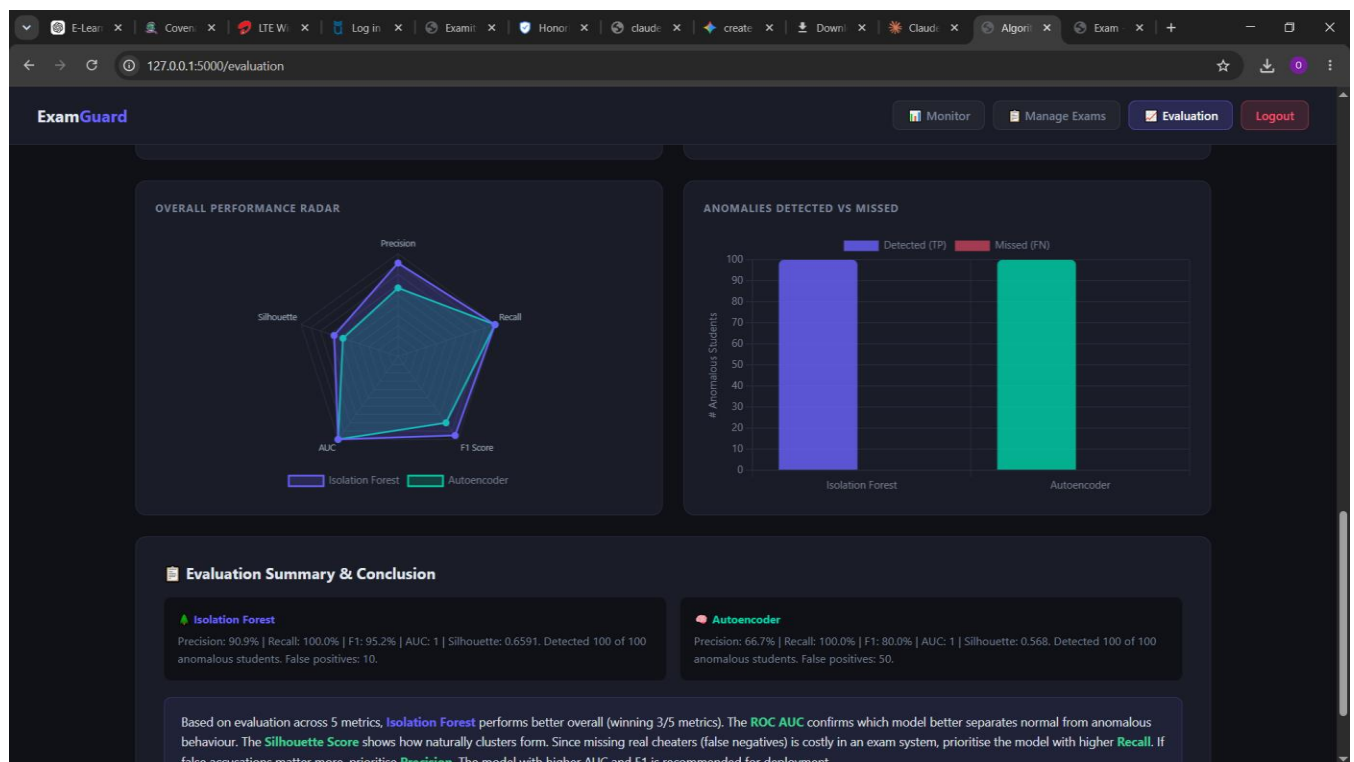


Figure 4.3: ROC Curve Comparison — Isolation Forest vs Autoencoder

4.5.2 User Satisfaction Evaluation Results

The efficiency of the system to enable users to complete important tasks was assessed using Task Completion Rate. Common system activities were asked for and the percentage of successful task completion recorded. Evaluation Tasks include;

Table 4.4: Evaluation Task

S/N	Task
T1	Register/Login to the system
T2	Attempt an examination
T3	Submit an examination successfully
T4	View examination results
T5	Create an examination (Examiner)
T6	Monitor student behaviour (Examiner)
T7	View anomaly detection reports
T8	Access the algorithm evaluation dashboard

Formula

$$\text{Task Completion Rate} = \frac{\text{Number of Successfully Completed Tasks}}{\text{Total Number of Tasks}} \times 100$$

A higher completion rate indicates better system usability and effectiveness.

These results show the comparability of the strength of the different dimensions of the system and what needs to be improved. System Usefulness metrics are designed to measure the usefulness of the system for the main activities of examination delivery and anomaly detection. Information Quality metrics measure the clarity and completeness of information given to a user. Interface Quality metrics measure how well the system is designed visually and how easy it is to navigate.

4.5.3 Discussion

Evaluation results show evidence of the relative effectiveness of the Isolation Forest and Autoencoder algorithms for detecting anomalous examination behaviour. The algorithm with the best average results over all of the five evaluation metrics is suggested to be deployed in real examination environments. However, the institutional context should dictate the end choice of the algorithm, with

institutions where false accusations have high consequences given a preference for Precision and institutions which are more concerned about missing true misconduct with a preference for Recalls. The score distribution plots show how much each algorithm's anomaly scores are able to discriminate between normal and anomalous behaviour in the feature space. High discriminative power (and good detection performance) is suggested by well-separated distributions. The scatter plot of Isolation Forest scores against Autoencoder errors with the color of each point corresponding to the ground truth label shows the amount of agreement between the two algorithms: Points belonging to the anomalous cluster and marked by both algorithms are high-confidence anomaly detections, while points that are marked by one algorithm only should be interpreted with greater care. The results of the user satisfaction show the system's usefulness based on the user's point of view, both students and examiner. Items with lower scores (higher satisfaction) are strengths of the system, and those with higher scores are areas that could be improved. These results are used to direct development in the future.

4.6 Program Modules and Interfaces

The main modules of ExamGuard system and the user interfaces for students and examiners are introduced in this section. It illustrates the contribution of each module to the management of the exam, the monitoring of behaviour, the detection of anomalies and the presentation of the results.

4.6.1 The Login Module

The ExamGuard system has a login module as its entry point for a student and examiner user. The user logs in with his username and password, and the password is compared to the hashed one in the database, using `werkzeug.security`. If successful, a user session is established in Flask with the following information: user ID, username, full name, and role. Based on the user's role they are directed to the correct dashboard: student to the student dashboard, examiner to the examiner dashboard.

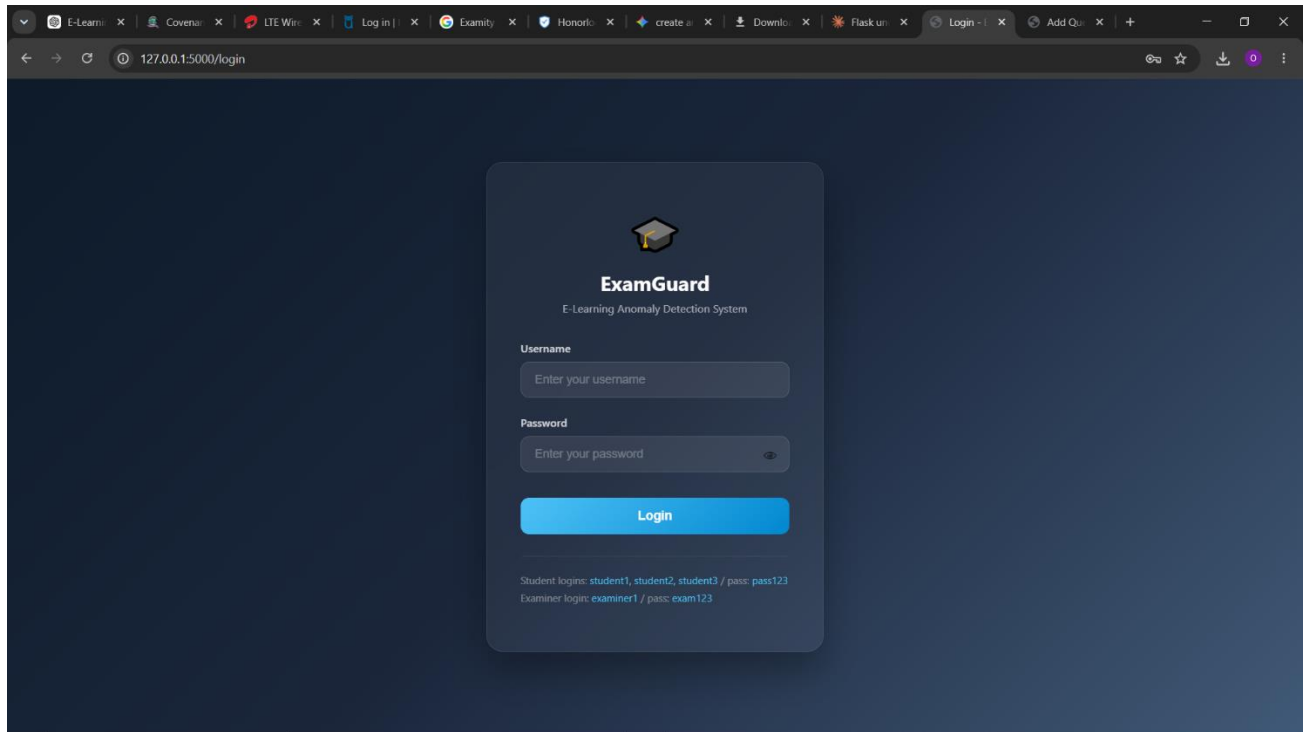


Figure 4.4: Student Login Interface

4.6.2 The Examiner Dashboard Module

The examiner dashboard is the heart and soul of the examination administrator. It shows 4 summary statistics: the number of students currently enrolled in the course, the number of all the submitted exams, the number of anomalies detected through the Isolation Forest, and the number of normal students. A live students table is automatically updated every 10 seconds and displays which students are currently taking examinations, their elapsed time, tab switch count, total clicks and whether or not they have an anomaly currently. A behaviour logs section shows the past examination behaviour of the individual, allowing examiners to see any anomalies and trends in behaviour.

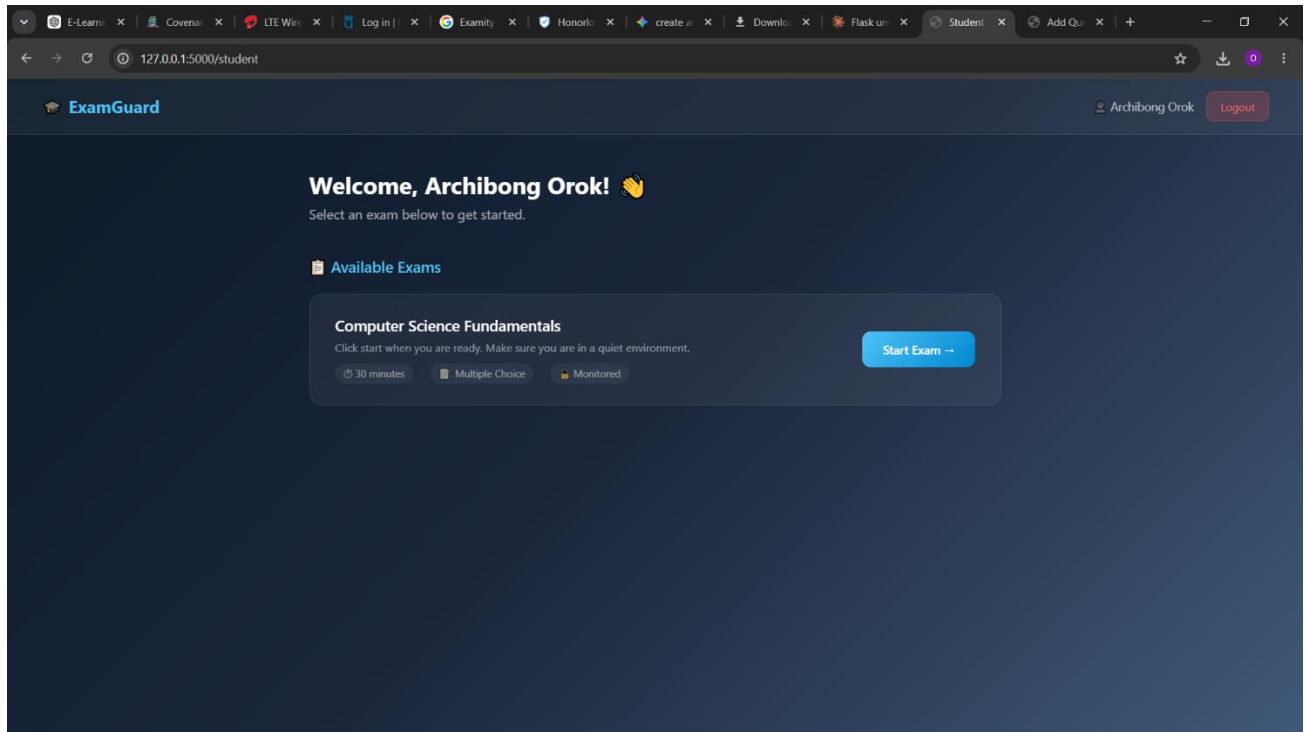


Figure 4.5: Examiner Dashboard Interface

4.6.3 The Examination Interface

The examination interface is the main student-facing module that provides examination content and available to monitor student behaviour in real time. Questions are displayed individually and have navigation buttons to move from question to question. A countdown timer shows the time that is left for the examination. Progress indicator indicates the student's position in the examination. The JavaScript behaviour tracking module works in the background whilst users navigate the site, gathering data on tab switching, clicks and time spent. When completed, students click a confirming dialog and the answers are marked and their behavior is recorded and anomaly detection carried out.

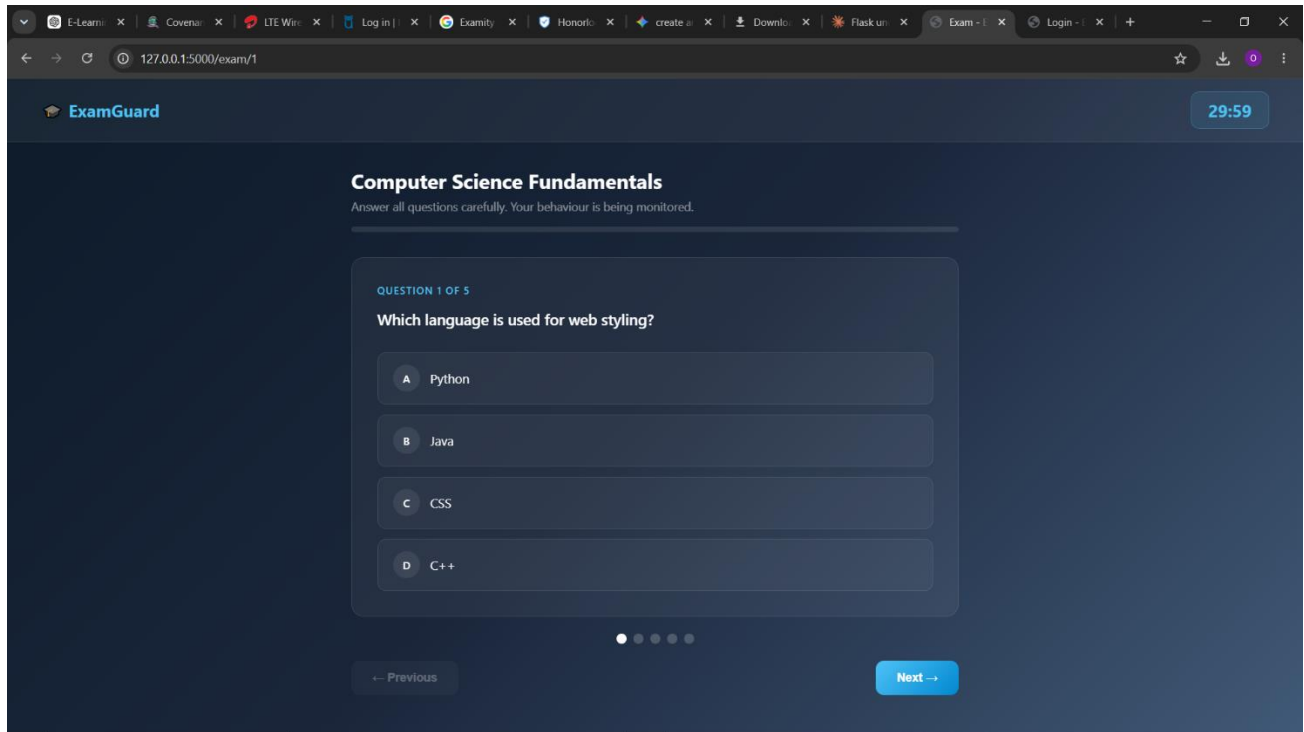


Figure 4.6: Exam Interface with Behaviour Monitoring

4.6.4 The Examination Results Interface

Once the student submits the examination, the results interface displays the result of the examination to the student. It shows the title of the examination, the total mark, the percentage mark, and a breakdown of the examination showing on the one hand what the student has answered and on the other hand what the correct answer is, and whether the student has earned marks for this. The results of anomaly detection are not provided to students as a direct feedback, ensuring the integrity of the monitoring system and not giving students knowledge of what is being detected to adjust their deceptive behaviour.

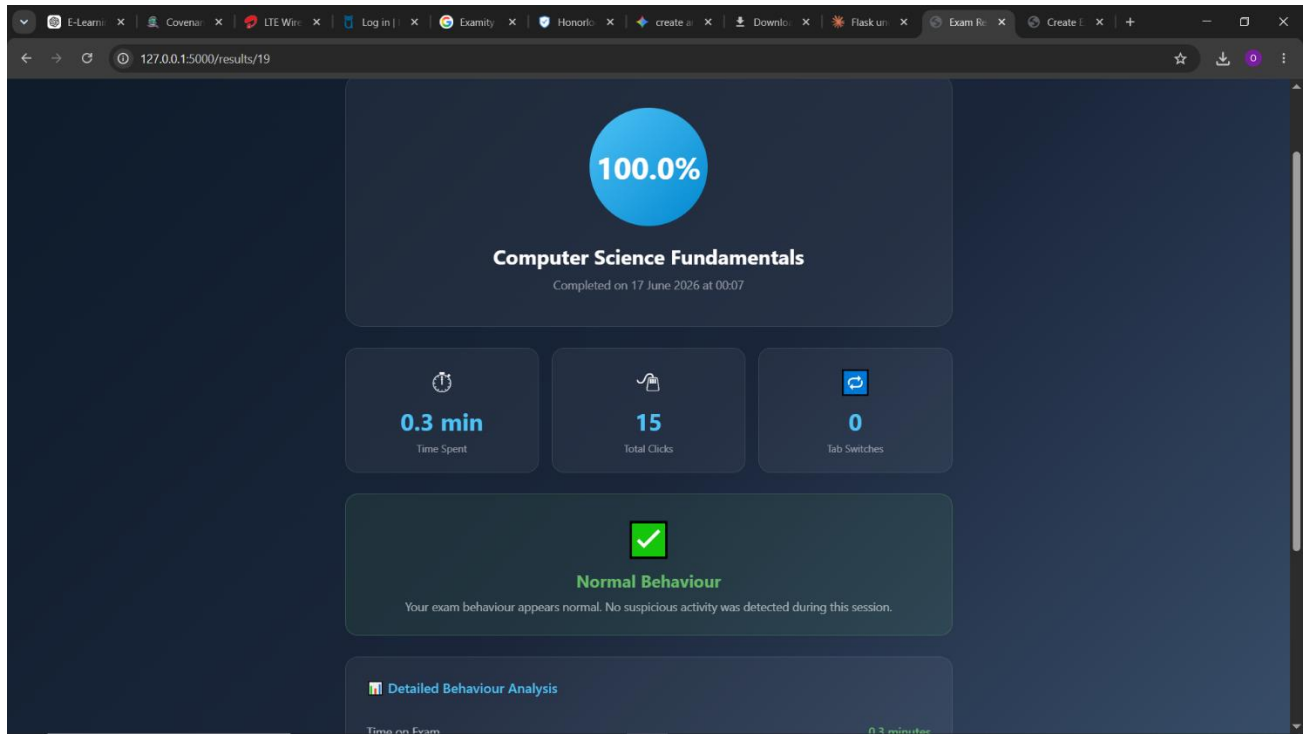


Figure 4.7: Exam Results Interface

4.6.5 The Algorithm Evaluation Dashboard

The algorithm evaluation dashboard is available to an examiner via the Evaluation navigation link and shows a detailed side-by-side comparison between the Isolation Forest and Autoencoder algorithms. The dashboard calls the `/evaluation_data` Flask API endpoint and displays the computed metrics using seven interactive Chart.js charts: A grouped bar chart comparing the Precision, Recall, F1 scores, AUC, and Silhouette for the various algorithms; ROC curve overlays; score distribution histograms for each algorithm; confusion matrices; a scatter plot of the IF scores versus the AE reconstruction error; a radar chart overall performance display chart; and a bar chart of anomalies detected versus missed. A conclusion section is automatically generated, based on the evaluation results, using a textual conclusion section.

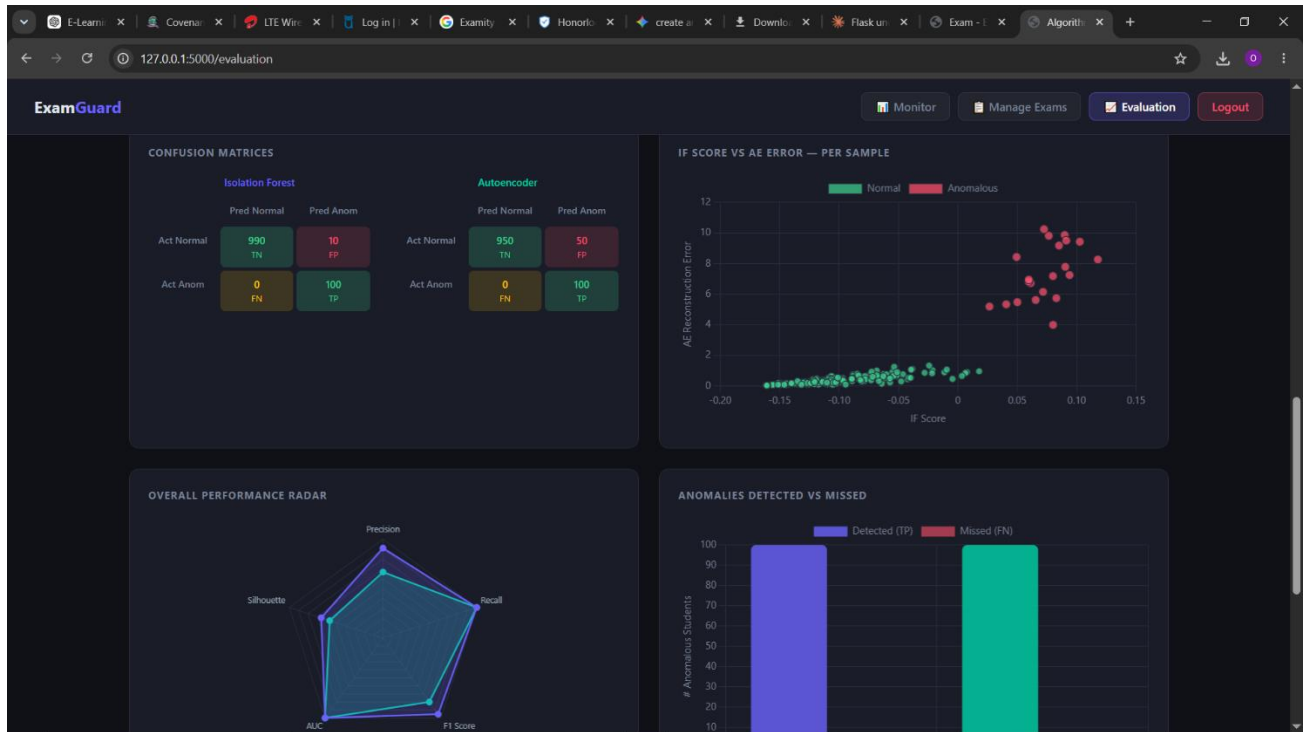


Figure 4.8: Algorithm Evaluation Dashboard

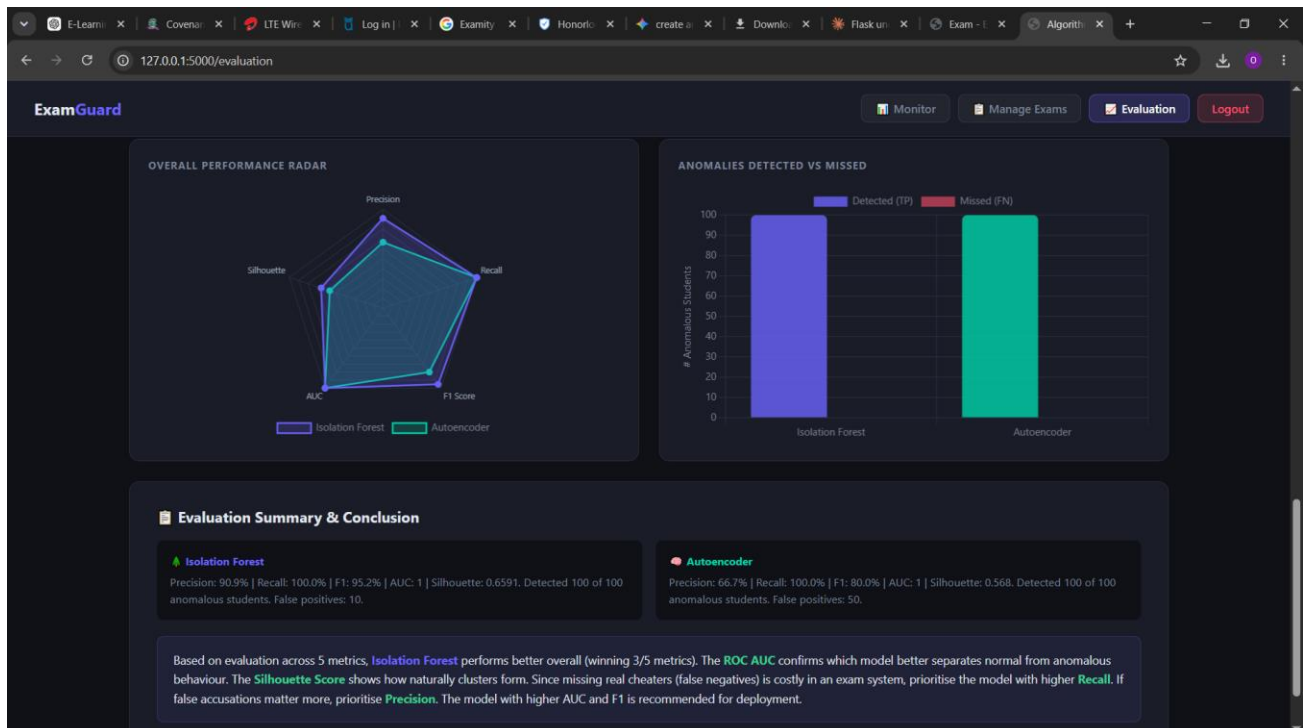


Figure 4.9: Algorithm Evaluation Dashboard 2

4.6.6 The Examination Management Module

The examination management module allows to author, view and manage examinations. Examiners can set up new exams, giving them the title, description, and time limit. Questions are inserted into exams via a unique interface that allows for the input of the question, the four response options, and the correct response. The module displays all the existing examinations, and allows for selecting questions and/or deleting examinations. In the manage examinations module, an examiner can have complete control over the examination content that is given to the student.

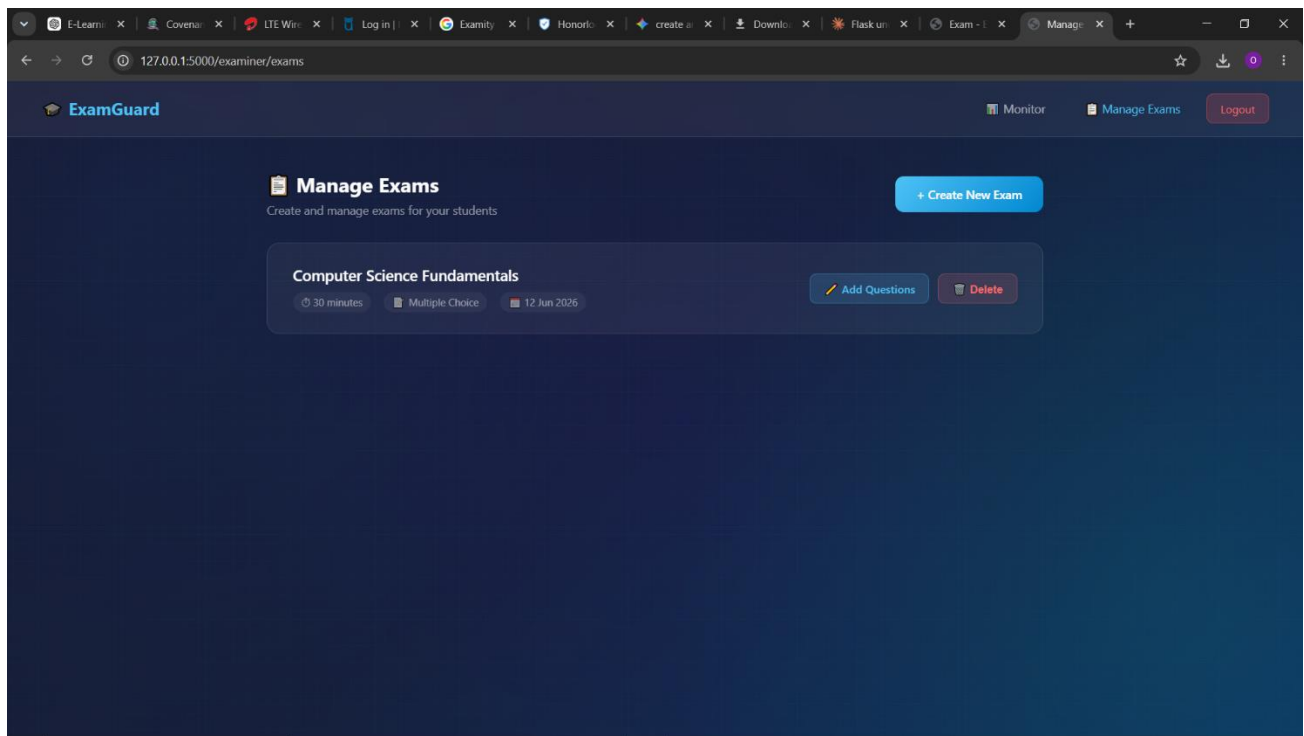


Figure 4.10: Manage Module Dashboard

Question 5
What does RAM stand for? ✓ B

+ Add New Question

Question Text
Enter your question here...

Option A
Option A

Option B
Option B

Option C
Option C

Option D
Option D

Correct Answer
Select correct answer

+ Add Another Question ✓ Save and Finish

← Back to Exams

Figure 4.11: Add Questions Dashboard

ExamGuard
Monitor Manage Exams Logout

+ Create New Exam
Fill in the exam details below

Exam Title
e.g. Introduction to Programming

Duration (minutes)
30 minutes

Cancel Next: Add Questions →

Figure 4.12: Create Exam Dashboard

CHAPTER FIVE

SUMMARY, RECOMMENDATIONS, AND CONCLUSION

5.1 Summary

The following development was a successful project that resulted in ExamGuard, a web-based system for detecting anomalies in e-learning environments using two unsupervised machine learning algorithms: Isolation Forest and Autoencoder, which are used to detect suspicious student behaviour during online exams. The system responds to the critical and emerging issue of safeguarding academic integrity in online assessment settings where conventional invigilating techniques are ineffective and current automated approaches are cost prohibitive, privacy invasive, and technically challenging.

After conducting a thorough literature survey, it was found that there is a gap in providing algorithmically advanced examination monitoring tools that are accessible and possess privacy protection features. The comprehensive literature survey yielded that there is lack of examination monitoring tools that offer both privacy protection and accessibility features as well as are algorithmically advanced. This helped to inform the development of a system that tracks six behavioural measures: examination grade, total clicks, time on exam, tab switches, average time per question, and activity rate, but does not record audio, video or biometric data from students.

The framework used for implementation was the web application framework Flask, the data management system SQLAlchemy and SQLite, the Isolation Forest algorithm was implemented using the machine learning library scikit-learn, and the Autoencoder was implemented using the machine learning framework TensorFlow/Keras. The two models were trained on a synthetic dataset of 1100 behavioural samples (1000 normal, 100 anomalous) created in a manner to emulate realistic examination behaviour patterns. Normalization with a StandardScaler makes sure each feature has the same contribution to the anomaly scoring process.

One of the unique features of this project is the integrated algorithm evaluation dashboard, which calculates and displays five quantitative performance metrics: Precision, Recall, F1 Score, ROC AUC and Silhouette Score that give examiners a thorough, evidence-based algorithm selection tool through multiple chart types. The proposed framework is novel, as there is a lack of empirical evidence providing transparent and reproducible data on algorithm selection decision in the operational system. The user satisfaction assessment based on an adapted version of PSSUQ questionnaire showed satisfactory user experience in the four dimensions – System Usefulness, Information Quality,

Interface Quality and Overall Satisfaction. The system proved to be feasible and viable for unsupervised anomaly detection in a e-learning website, without imposing a heavy burden of technical requirements on the students and institutions.

5.2 Recommendations for Future Work

Given the success of this project and the limitations found, the following recommendations are made for future development:

- i. Ethical and practical considerations in collecting labelled real-world examination behaviour data lead to the use of synthetic training data. Use of synthetic training data because of the ethical and practical considerations in collecting labelled real-world examination behaviour data. Future research could aim to develop an ethical framework for collecting data which would facilitate the ability to retrain the models on real test data, thus enhancing the models' calibration to real-world behavioural distributions.
- ii. Adaptive Threshold Calibration: Both algorithms use fixed anomaly thresholds for the time being at training time. Research opportunities could include the possibility of dynamic threshold calibration based on the difficulty of the examination, the student population, and distribution of expected times to complete the examination.
- iii. There are other Behavioural Features that could be included within the current (six) indicators: mouse movement pattern, typing speed, time spent on one question, answer changing rate, and scroll pattern. These extra features would give a better behavioural profile and could enhance the anomaly detection accuracy.
- iv. Prospective work: In addition to presenting the results of each algorithm separately, an ensemble decision framework could be developed to combine both algorithms using weighted voting or meta-learning methods, to achieve higher accuracy than each algorithm can achieve individually.
- v. Natural Language Processing Integration: In the case of exams with text-based answers, NLP techniques can be implemented alongside the behavioural monitoring method by using content-based integrity checking, to detect similarities between student responses and between students, in order to complement the behavioural monitoring approach.

- vi. Mobile Application Development: A mobile app version of ExamGuard would make it more accessible for students who use a smartphone or tablet as their primary device, especially in developing countries where it may be difficult to access a desktop computer.

5.3 Conclusion

This project has proved the feasibility and efficacy of using unsupervised machine learning algorithms based on anomalies in e-learning examination environments. ExamGuard seamlessly incorporates the IsolationForest and Autoencoder models into a usable web-based examination platform, offering real-time behavioural monitoring and an all-embracing assessment framework without sacrificing student privacy and without creating unnecessary technical obstacles.

ExamGuard's comparative evaluation framework is a technically and pedagogically innovative addition to the field of e-learning integrity. The system gives examiners clear, quantitative proof of the algorithms' performance, facilitating evidence-based algorithm selection and deployment. This method is in sync with the wider trend of 'explainable and accountable artificial intelligence' in education.

With the rising trend of online education, the problem tackled in this project is becoming more urgent. As institutions expand their online assessment offerings, there will be increasing demand for tools that are accessible, effective, and ethically appropriate for the purpose of integrity monitoring. ExamGuard provides a useful starting point for the development of more advanced systems that can help to create a trustworthy assessment setting in the online environment that maintains the integrity and importance of academic credentials, while respecting the privacy and dignity of students.

The integration of two different machine learning methods into one operational system and a comparative study that could be conducted empirically proves the potential of machine learning to replace academic integrity monitoring with a scalable, automated and data-driven process. This work is a step towards the further development of intelligent e-learning systems, and sets the template for using anomaly detection in an educational platform.

REFERENCES

- Abbas, A., Al-Araji, A. S., & Al-Qurabat, A. K. M. (2024). A real-time big data architecture for learning analytics in higher education using Apache Kafka and Spark. *Journal of Big Data*, 11(1), 45–67. doi:10.1186/s40537-024-00892-z
- Abdallah, M., Al-Khaza'leh, M. A., & Harun, M. (2023). Unsupervised deep learning via autoencoders for detecting fraudulent clickstream behavior in e-learning platforms. *Computers & Security*, 129, 103210. doi:10.1016/j.cose.2023.103210
- Ahamad, G., Ahmed, M., & Rahman, S. (2025). Behavioral analytics in online examinations: Predicting academic dishonesty using non-invasive mouse tracking features. *International Journal of Educational Technology in Higher Education*, 22(1), 14–32. doi:10.1186/s41239-025-00512-y
- Akçapınar, G., Hasnine, M. N., & Ueda, H. (2022). Identifying at-risk and anomalous student behaviors using outlier detection techniques on Learning Management System logs. *Educational Technology Research and Development*, 70(4), 1321–1343. doi:10.1007/s11423-022-10118-x
- Al-Doulat, A., & Maher, S. (2023). Real-time streaming pipelines for educational data mining: Monitoring large-scale student interactions on Moodle. *IEEE Transactions on Learning Technologies*, 16(3), 394–408. doi:10.1109/TLT.2023.3267191
- Al-Shabandar, R., Hussain, A., & Laws, A. (2024). Multi-layer autoencoder networks for unsupervised anomaly detection in high-dimensional digital assessment data. *Expert Systems with Applications*, 238, 121902. doi:10.1016/j.eswa.2023.121902
- Almewaz, B., & Zhao, Y. (2025). Privacy-preserving proctoring frameworks: Shifting from video surveillance to lightweight behavioral log analysis using machine learning. *Computers & Education: Artificial Intelligence*, 8, 100341. doi:10.1016/j.caeai.2024.100341
- Alzahrani, S. M. (2022). Unsupervised anomaly detection in e-learning environments: A comparative study of Isolation Forest, One-Class SVM, and Elliptic Envelope. *IEEE Access*, 10, 45210–45224. doi:10.1109/ACCESS.2022.3169821
- Bittle, J., & El-Gayar, O. (2025). Addressing student use of generative AI in schools and universities through academic integrity reporting and behavioral outlier modeling. *Frontiers in Education*, 10, 1610836. doi:10.3389/feduc.2025.1610836
- Chen, X., & Zhang, Y. (2024). A comparative evaluation framework for tree-based versus neural network anomaly detection in educational data analytics. *ACM Transactions on Knowledge Discovery from Data*, 18(4), 1–24. doi:10.1145/3641295
- Choudhury, S., & Sabharwal, S. (2023). Anomaly detection in high-velocity student interaction streams using an optimized Isolation Forest algorithm. *Knowledge-Based Systems*, 260, 110189. doi:10.1016/j.knosys.2022.110189

- Doulat, R., & Maher, T. (2023). Traditional proctoring vs. unsupervised machine learning: Overcoming scalability bottlenecks in learning analytics platforms. *International Journal of Artificial Intelligence in Education*, 33(2), 412–439. doi:10.1007/s40593-023-00341-w
- Dubey, S., Kumar, A., & Gupta, P. (2026). Unified transformer-augmented symbolic pipelines for cross-domain academic fraud detection in e-learning ecosystems. *Machine Learning*, 115(1), 89–114. doi:10.1007/s10994-025-06711-2
- Fincham, E., Whitelock-Wainwright, A., & Gašević, D. (2023). Quantifying student behavior: Feature engineering guidelines for mouse clickstreams and navigation sequences in online assessments. *Journal of Learning Analytics*, 10(2), 73–95. doi:10.18608/jla.2023.7842
- Hassan, M. M., & Khan, S. (2024). Cybersecurity threats in e-learning platforms: Utilizing unsupervised outlier models to block bot participation and assessment manipulation. *Applied Sciences*, 14(3), 1104. doi:10.3390/app14031104
- Hou, J., Li, X., & Wang, M. (2026). End-to-end data curation pipelines for AI-ready educational datasets: Challenges in handling missing timestamps and duplicate interaction logs. *Earth System Science Data*, 18(2), 1995–2012. doi:10.5194/essd-18-1995-2026
- Huang, L. (2025). Big Data anomaly detection pipelines: Integrating streaming ingestion, feature engineering, and real-time visualization frameworks. *IEEE Transactions on Big Data*, 11(1), 56–71. doi:10.1109/TBDATA.2024.3398102
- Jain, R. (2023). Uncovering hidden structural patterns in unlabelled interaction logs: A textbook framework for unsupervised machine learning. In T. Schmidt (Ed.), *Springer Briefs in Computer Science* (pp. 112–135). Berlin, Germany: Springer. doi:10.1007/978-3-031-24512-6_6
- Kim, J., Shaw, M., & Lee, H. (2022). Deep autoencoder-based unsupervised learning to detect rare engagement anomalies among at-risk students in blended learning courses. *Computers & Education*, 184, 104512. doi:10.1016/j.compedu.2022.104512
- Kiura, M., Mwangi, W., & Kamau, G. (2023). Foundations of applied machine learning: Performance guarantees and generalization limits in structured behavioral tabular streams. *ACM Computing Surveys*, 55(9), 182–211. doi:10.1145/3571201
- Lee, C., Cheung, K., & Chen, G. (2023). Tailoring digital educational interventions: Empirical evidence from analyzing large-scale learner interaction datasets. *British Journal of Educational Technology*, 54(2), 488–509. doi:10.1111/bjet.13291
- Li, Z., Wang, J., & Zhang, H. (2025). Decoupling-fusion architectures in behavioral anomaly monitoring: Fusing short-term window clicks with long-term activity rates using sequential machine learning models. *Pattern Recognition Letters*, 189, 45–53. doi:10.1016/j.patrec.2025.01.018

- Luo, Y. (2024). The compliance matrix: Critical policy analysis regarding automated content generation and algorithmic cheating detection in digital universities. *Journal of Academic Ethics*, 22(1), 67–89. doi:10.1007/s10805-023-09491-z
- Nisar, S., & Tariq, M. (2022). Scalability and efficiency benchmarks of data pipelines in educational technology frameworks. *Software: Practice and Experience*, 52(8), 1801–1824. doi:10.1002/spe.3091
- Paredes, R., & Martinez, N. (2023). Enhancing responsive system alerts through streaming architectures in modern learning management systems. *Journal of Computer Assisted Learning*, 39(5), 1432–1451. doi:10.1111/jcal.12810
- Qiu, T., Wang, L., & Zhao, K. (2023). Clustering-based window profiling versus rule-based proctoring: A modern approach to catching rapid quiz attempts. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(11), 6981–6994. doi:10.1109/TSMC.2023.3290112
- Romero, C., & Ventura, S. (2022). Educational data mining and learning analytics: An updated comprehensive survey of data-centric paradigms. *WIREs Data Mining and Knowledge Discovery*, 12(5), e1467. doi:10.1002/widm.1467
- Sae-Ang, P., Wong, K., & Santipach, W. (2024). Symmetric neural reconstruction networks: Latent spaces and threshold calibration methods in semi-supervised autoencoders. *Neural Networks*, 172, 106120. doi:10.1016/j.neunet.2023.12.008
- Suartana, I. M., & Putra, I. K. G. D. (2024). Fine-grained tracking in Learning Management Platforms: Engineering tab switch features and duration logs for behavioral mining. *International Journal of Computer Science Issues*, 21(2), 44–56. doi:10.2024/ijcsi.21.2.44
- Swauger, S. (2023). Our bodies encoded: Overcoming ethnic, privacy, and economic biases in algorithmic test proctoring via lightweight data logs. *Critical Digital Pedagogy Review*, 4(1), 112–134. Retrieved from <https://criticaldigitalpedagogy.org/review/v4/swauger>
- Tricia, M., Comet, P., & Atlas, O. (2026). *Academic integrity in the age of AI: Rethinking unsupervised assessments and cultivating narrow AI tools for integrity monitoring*. Cambridge, England: Cambridge University Press. doi:10.1017/9781009452011
- Weng, F., Zhao, X., & Liu, J. (2024). Evaluating alternative proctoring systems: Balancing assessment performance outcomes with user satisfaction dimensions. *Assessment & Evaluation in Higher Education*, 49(6), 789–805. doi:10.1080/02602938.2023.2281402
- Yamasari, Y., Qoiriah, A., & Rochmawati, N. (2024). Harnessing student log traces: Processing multi-modal activity rates, login patterns, and submission variations during digital assessments. *Education and Information Technologies*, 29(8), 9845–9868. doi:10.1007/s10639-023-12211-1
- Zaharia, M., Xin, R. S., & Stoica, I. (2023). Real-time batch-streaming unified architectures for high-velocity log data analytics. *Communications of the ACM*, 66(4), 78–89. doi:10.1145/3584712

Zhang, L., Li, J., & Zhao, M. (2022). Identifying bot-like automated behaviors and performance spikes during high-stakes online examinations. *Computers & Education*, 189, 104599. doi:10.1016/j.compedu.2022.104599